

```
In [56]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

# Setting untuk membuat angka mudah dibaca di display
pd.options.display.float_format = '{:20.2f}'.format

# Menampilkan semua kolom pada output
pd.set_option('display.max_columns', None)
```

Data Exploration

```
In [57]: df = pd.read_excel("../data/online_retail_II.xlsx", sheet_name=0) # sheet name buat
df.head(10)
```

Out[57]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13085.00	United Kingdom
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.00	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.00	United Kingdom
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13085.00	United Kingdom
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13085.00	United Kingdom
5	489434	22064	PINK DOUGHNUT TRINKET POT	24	2009-12-01 07:45:00	1.65	13085.00	United Kingdom
6	489434	21871	SAVE THE PLANET MUG	24	2009-12-01 07:45:00	1.25	13085.00	United Kingdom
7	489434	21523	FANCY FONT HOME SWEET HOME DOORMAT	10	2009-12-01 07:45:00	5.95	13085.00	United Kingdom
8	489435	22350	CAT BOWL	12	2009-12-01 07:46:00	2.55	13085.00	United Kingdom
9	489435	22349	DOG BOWL , CHASING BALL DESIGN	12	2009-12-01 07:46:00	3.75	13085.00	United Kingdom

In [58]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 525461 entries, 0 to 525460
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Invoice          525461 non-null object
1   StockCode       525461 non-null object
2   Description      522533 non-null object
3   Quantity        525461 non-null int64
4   InvoiceDate      525461 non-null datetime64[ns]
5   Price           525461 non-null float64
6   Customer ID     417534 non-null float64
7   Country         525461 non-null object
dtypes: datetime64[ns](1), float64(2), int64(1), object(4)
memory usage: 32.1+ MB
```

In [59]: `df.describe()`

Out[59]:

	Quantity	InvoiceDate	Price	Customer ID
count	525461.00	525461	525461.00	417534.00
mean	10.34	2010-06-28 11:37:36.845017856	4.69	15360.65
min	-9600.00	2009-12-01 07:45:00	-53594.36	12346.00
25%	1.00	2010-03-21 12:20:00	1.25	13983.00
50%	3.00	2010-07-06 09:51:00	2.10	15311.00
75%	10.00	2010-10-15 12:45:00	4.21	16799.00
max	19152.00	2010-12-09 20:01:00	25111.09	18287.00
std	107.42	NaN	146.13	1680.81

Berdasarkan gambaran besar yang kita lakukan disini, ada beberapa hal yang bikin janggal:

- Nilai min pada **Quantity** bernilai negatif yang dimana itu cukup aneh untuk sekilas, karena kita berekspektasi untuk positif
- Nilai min pada **Price** juga bernilai negatif
- Jumlah **Customer ID** yang jumlahnya kurang dibandingkan dengan kolom lainnya

Kita akan investigasi itu kedepannya

In [60]: `df.describe(include='O') # Mendapatkan deskripsi untuk objek data type (string)`

Out[60]:

	Invoice	StockCode	Description	Country
count	525461	525461	522533	525461
unique	28816	4632	4681	40
top	537434	85123A	WHITE HANGING HEART T-LIGHT HOLDER	United Kingdom
freq	675	3516	3549	485852

In [61]: `df[df['Customer ID'].isna()].head(10)`

Out[61]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
263	489464	21733	85123a mixed	-96	2009-12-01 10:52:00	0.00	NaN	United Kingdom
283	489463	71477	short	-240	2009-12-01 10:52:00	0.00	NaN	United Kingdom
284	489467	85123A	21733 mixed	-192	2009-12-01 10:53:00	0.00	NaN	United Kingdom
470	489521	21646	NaN	-50	2009-12-01 11:44:00	0.00	NaN	United Kingdom
577	489525	85226C	BLUE PULL BACK RACING CAR	1	2009-12-01 11:49:00	0.55	NaN	United Kingdom
578	489525	85227	SET/6 3D KIT CARDS FOR KIDS	1	2009-12-01 11:49:00	0.85	NaN	United Kingdom
1055	489548	22271	FELTCRAFT DOLL ROSIE	1	2009-12-01 12:32:00	2.95	NaN	United Kingdom
1056	489548	22254	FELT TOADSTOOL LARGE	12	2009-12-01 12:32:00	1.25	NaN	United Kingdom
1057	489548	22273	FELTCRAFT DOLL MOLLY	3	2009-12-01 12:32:00	2.95	NaN	United Kingdom
1058	489548	22195	LARGE HEART MEASURING SPOONS	1	2009-12-01 12:32:00	1.65	NaN	United Kingdom

Hmm, kalau diliat liat, pada tabel tersebut, untuk data pertama hingga ke empat keliatan aneh sekali. Disitu **Price** = 0 dan **Quantity** bernilai negatif. Lalu beberapa data meskipun disitu keliatan ada **Description** yang legit, namun kita tidak bisa secara effective mengetahui siapa **Customer ID** yang membeli ini.

Kemungkinan besar strategi kita saat ini adalah untuk menghapus **Customer ID** yang hilang/kosong

In [62]: `df[df["Quantity"] < 0].head(10) # Penasaran dengan apa sih nilai negatif pada data i`

Out[62]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
178	C489449	22087	PAPER BUNTING WHITE LACE	-12	2009-12-01 10:33:00	2.95	16321.00	Australia
179	C489449	85206A	CREAM FELT EASTER EGG BASKET	-6	2009-12-01 10:33:00	1.65	16321.00	Australia
180	C489449	21895	POTTING SHED SOW 'N' GROW SET	-4	2009-12-01 10:33:00	4.25	16321.00	Australia
181	C489449	21896	POTTING SHED TWINE	-6	2009-12-01 10:33:00	2.10	16321.00	Australia
182	C489449	22083	PAPER CHAIN KIT RETRO SPOT	-12	2009-12-01 10:33:00	2.95	16321.00	Australia
183	C489449	21871	SAVE THE PLANET MUG	-12	2009-12-01 10:33:00	1.25	16321.00	Australia
184	C489449	84946	ANTIQUE SILVER TEA GLASS ETCHED	-12	2009-12-01 10:33:00	1.25	16321.00	Australia
185	C489449	84970S	HANGING HEART ZINC T-LIGHT HOLDER	-24	2009-12-01 10:33:00	0.85	16321.00	Australia
186	C489449	22090	PAPER BUNTING RETRO SPOTS	-12	2009-12-01 10:33:00	2.95	16321.00	Australia
196	C489459	90200A	PURPLE SWEETHEART BRACELET	-3	2009-12-01 10:44:00	4.25	17592.00	United Kingdom

Additional Variable Information

InvoiceNo: Invoice number. Nominal. A 6-digit integral number uniquely assigned to each transaction. If this code starts with the letter 'c', it indicates a cancellation.

StockCode: Product (item) code. Nominal. A 5-digit integral number uniquely assigned to each distinct product.

Description: Product (item) name. Nominal.

Quantity: The quantities of each product (item) per transaction. Numeric.

InvoiceDate: Invoice date and time. Numeric. The day and time when a transaction was generated.

UnitPrice: Unit price. Numeric. Product price per unit in sterling (£).

CustomerID: Customer number. Nominal. A 5-digit integral number uniquely assigned to each customer.

Country: Country name. Nominal. The name of the country where a customer resides.

SHOW LESS 

Ternyata kalau kita lihat deskripsi pada data asli ini, maka dapat diketahui bahwa kode unik C berarti Cancellation

Cukup menarik untuk di tinjau lebih lanjut

```
In [63]: df["Invoice"] = df["Invoice"].astype("str")
df[df["Invoice"].str.match("^\\d{6}$") == False] # Maksud kodingan ini adalah, kita
```

Out[63]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
178	C489449	22087	PAPER BUNTING WHITE LACE	-12	2009-12-01 10:33:00	2.95	16321.00	Austral
179	C489449	85206A	CREAM FELT EASTER EGG BASKET	-6	2009-12-01 10:33:00	1.65	16321.00	Austral
180	C489449	21895	POTTING SHED SOW 'N' GROW SET	-4	2009-12-01 10:33:00	4.25	16321.00	Austral
181	C489449	21896	POTTING SHED TWINE	-6	2009-12-01 10:33:00	2.10	16321.00	Austral
182	C489449	22083	PAPER CHAIN KIT RETRO SPOT	-12	2009-12-01 10:33:00	2.95	16321.00	Austral
...	...	...	...	...	...	...	...	...
524695	C538123	22956	36 FOIL HEART CAKE CASES	-2	2010-12-09 15:41:00	2.10	12605.00	German
524696	C538124	M	Manual	-4	2010-12-09 15:43:00	0.50	15329.00	United Kingdom
524697	C538124	22699	ROSES REGENCY TEACUP AND SAUCER	-1	2010-12-09 15:43:00	2.95	15329.00	United Kingdom
524698	C538124	22423	REGENCY CAKESTAND 3 TIER	-1	2010-12-09 15:43:00	12.75	15329.00	United Kingdom
525282	C538164	35004B	SET OF 3 BLACK FLYING DUCKS	-1	2010-12-09 17:32:00	1.95	14031.00	United Kingdom

10209 rows × 8 columns



```
In [64]: df["Invoice"].str.replace("[0-9]", "", regex=True).unique() # Maksudnya yaitu "ToLo
```

```
Out[64]: array(['', 'C', 'A'], dtype=object)
```

```
In [65]: df[df["Invoice"].str.startswith("A")]
```

Out[65]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Cc
179403	A506401	B	Adjust bad debt	1	2010-04-29 13:36:00	-53594.36	NaN	I Kir
276274	A516228	B	Adjust bad debt	1	2010-07-19 11:24:00	-44031.79	NaN	I Kir
403472	A528059	B	Adjust bad debt	1	2010-10-20 12:04:00	-38925.87	NaN	I Kir

Aman untuk dibersihkan saja, karena dari deskripsi nya dan juga harganyaaa tidak masuk akal

Tapi ini maksudnya apa??

di dunia bisnis/akuntansi, ini adalah hal yang sangat nyata dan penting.

Nama resminya di dunia nyata (Akuntansi/Bisnis) adalah **"Bad Debt Write-off"** atau dalam bahasa Indonesia **"Penghapusan Piutang Tak Tertagih"**.

Cerita Kasus "Bad Debt"

1. **Awal Cerita:** Toko mengirim barang ke pembeli besar. Karena langganan, pembeli bilang "Kirim dulu, nanti aku bayar bulan depan". Toko mencatat ini sebagai **Pendapatan (Sales)** sebesar misal £50.000.
2. **Masalah:** Bulan depan datang, si pembeli tidak bayar. Bulan depannya lagi, ditagih susah. Ternyata pembelinya bangkrut atau kabur.
3. **Realita:** Uang £50.000 itu **tidak akan pernah diterima** oleh toko.
4. **Solusi Akuntansi (Adjust Bad Debt):** Toko tidak boleh membiarkan catatan "Pendapatan £50.000" itu ada di buku, karena nanti toko harus bayar pajak atas uang yang tidak pernah mereka terima.

Makanya, bagian keuangan membuat **entri manual** (seperti yang kamu lihat di data itu):

- **Description:** "Adjust bad debt" (Koreksi utang macet).

- **Price:** Dibuat **Negatif** (misal -53594.36) untuk "membatalkan" atau "mengurangi" total pendapatan toko secara keseluruhan.
- **StockCode 'B':** Kemungkinan kode manual untuk 'Bad Debt'.

Kenapa datanya aneh?

- **Quantity 1 tapi Harga Minus:** Ini cuma cara orang keuangan memasukkan data ke sistem. Mereka butuh satu baris transaksi (Qty 1) yang nilainya minus besar untuk menyeimbangkan pembukuan.
- **Customer ID NaN:** Karena ini seringkali tindakan "pembersihan buku" di akhir bulan/tahun oleh akuntan, mereka mungkin tidak memasukkannya ke akun customer spesifik, tapi sebagai kerugian umum toko.

Kesimpulan: Itu adalah catatan **Kerugian Toko** akibat di-PHP-in (tidak dibayar) oleh pembeli yang utangnya macet.

```
In [66]: df["StockCode"] = df["StockCode"].astype("str")
df[(df["StockCode"].str.match("^\\d{5}$") == False)] #buang yang Barang Normal (5 a
```

Out[66]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Cc
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.00	Kir
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.00	Kir
12	489436	48173C	DOOR MAT BLACK FLOCK	10	2009-12-01 09:06:00	5.95	13078.00	Kir
23	489436	35004B	SET OF 3 BLACK FLYING DUCKS	12	2009-12-01 09:06:00	4.65	13078.00	Kir
28	489436	84596F	SMALL MARSHMALLOWS PINK BOWL	8	2009-12-01 09:06:00	1.25	13078.00	Kir
...	...	...	...	...	...	...	...	...
525387	538170	84029E	RED WOOLLY HOTTIE WHITE HEART.	2	2010-12-09 19:32:00	3.75	13969.00	Kir
525388	538170	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	2	2010-12-09 19:32:00	3.75	13969.00	Kir
525389	538170	85232B	SET OF 3 BABUSHKA STACKING TINS	2	2010-12-09 19:32:00	4.95	13969.00	Kir
525435	538171	47591D	PINK FAIRY CAKE CHILDRENS APRON	1	2010-12-09 20:01:00	1.95	17530.00	Kir
525436	538171	47591B	SCOTTIES CHILDRENS APRON	2	2010-12-09 20:01:00	1.65	17530.00	Kir

80112 rows × 8 columns



StockCode: Product (item) code. Nominal. A 5-digit integral number uniquely assigned to each distinct product.

itu adalah katanya deskripsi yang disediakan oleh distributor, sehingga disini ada tidak kecocokan dengan dokumentasi data. Namun kalau dicek, sepertinya ini aman aman aja deh, tapi perlu untuk dicek lebih dalam

In [67]: `df[(df["StockCode"].str.match("^\\d{5}$") == False) & (df["StockCode"].str.match("^`

Out[67]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
89	489439	POST	POSTAGE	3	2009-12-01 09:28:00	18.00	12682.00	France
126	489444	POST	POSTAGE	1	2009-12-01 09:55:00	141.00	12636.00	United Kingdom
173	489447	POST	POSTAGE	1	2009-12-01 10:10:00	130.00	12362.00	Belgium
625	489526	POST	POSTAGE	6	2009-12-01 11:50:00	18.00	12533.00	Germany
735	C489535	D	Discount	-1	2009-12-01 12:11:00	9.00	15299.00	United Kingdom
...	...	...	...	...	...	...	...	...
524776	538147	M	Manual	1	2010-12-09 16:11:00	15.00	13090.00	United Kingdom
524887	538148	DOT	DOTCOM POSTAGE	1	2010-12-09 16:26:00	547.32	NaN	United Kingdom
525000	538149	DOT	DOTCOM POSTAGE	1	2010-12-09 16:27:00	620.68	NaN	United Kingdom
525126	538153	DOT	DOTCOM POSTAGE	1	2010-12-09 16:31:00	822.94	NaN	United Kingdom
525147	538154	DOT	DOTCOM POSTAGE	1	2010-12-09 16:35:00	85.79	NaN	United Kingdom

3098 rows × 8 columns

In [68]: `df[(df["StockCode"].str.match("^\\d{5}$") == False) & (df["StockCode"].str.match("^`

Out[68]: array(['POST', 'D', 'DCGS0058', 'DCGS0068', 'DOT', 'M', 'DCGS0004',
'DCGS0076', 'C2', 'BANK CHARGES', 'DCGS0003', 'TEST001',
'gift_0001_80', 'DCGS0072', 'gift_0001_20', 'DCGS0044', 'TEST002',
'gift_0001_10', 'gift_0001_50', 'DCGS0066N', 'gift_0001_30',
'PADS', 'ADJUST', 'gift_0001_40', 'gift_0001_60', 'gift_0001_70',
'gift_0001_90', 'DCGSSGIRL', 'DCGS0006', 'DCGS0016', 'DCGS0027',
'DCGS0036', 'DCGS0039', 'DCGS0060', 'DCGS0056', 'DCGS0059', 'GIFT',
'DCGSLBOY', 'm', 'DCGS0053', 'DCGS0062', 'DCGS0037', 'DCGSSBOY',
'DCGSLGIRL', 'S', 'DCGS0069', 'DCGS0070', 'DCGS0075', 'B',
'DCGS0041', 'ADJUST2', '47503J ', 'C3', 'SP1002', 'AMAZONFEE'],
dtype=object)

In [69]: `df[df["StockCode"].str.contains("DOT")]`

Out[69]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
2379	489597	DOT	DOTCOM POSTAGE	1	2009-12-01 14:28:00	647.19	NaN	United Kingdom
2539	489600	DOT	DOTCOM POSTAGE	1	2009-12-01 14:43:00	55.96	NaN	United Kingdom
2551	489601	DOT	DOTCOM POSTAGE	1	2009-12-01 14:44:00	68.39	NaN	United Kingdom
2571	489602	DOT	DOTCOM POSTAGE	1	2009-12-01 14:45:00	59.35	NaN	United Kingdom
2619	489603	DOT	DOTCOM POSTAGE	1	2009-12-01 14:46:00	42.39	NaN	United Kingdom
...	...	...	...	...	...	...	...	...
524272	538071	DOT	DOTCOM POSTAGE	1	2010-12-09 14:09:00	885.94	NaN	United Kingdom
524887	538148	DOT	DOTCOM POSTAGE	1	2010-12-09 16:26:00	547.32	NaN	United Kingdom
525000	538149	DOT	DOTCOM POSTAGE	1	2010-12-09 16:27:00	620.68	NaN	United Kingdom
525126	538153	DOT	DOTCOM POSTAGE	1	2010-12-09 16:31:00	822.94	NaN	United Kingdom
525147	538154	DOT	DOTCOM POSTAGE	1	2010-12-09 16:35:00	85.79	NaN	United Kingdom

736 rows × 8 columns



```
In [70]: # Simpan kode-kode aneh tadi ke dalam variabel biar gampang
kode_aneh = df[(df["StockCode"].str.match("^\d{5}$") == False) & (df["StockCode"].

# Kita filter data cuma yang kodenya aneh, lalu kita lihat Description-nya
# .value_counts() biar sekalian tau mana yang paling sering muncul
df[df["StockCode"].isin(kode_aneh)].groupby(["StockCode", "Description"]).size().re
```

Out[70]:

	StockCode	Description	Jumlah Muncul
32	POST	POSTAGE	862
30	M	Manual	850
29	DOT	DOTCOM POSTAGE	735
9	C2	CARRIAGE	136
10	D	Discount	100
8	BANK CHARGES	Bank Charges	59
33	S	SAMPLES	41
2	ADJUST	Adjustment by john on 26/01/2010 16	38
16	DCGS0058	MISO PRETTY GUM	30
3	ADJUST	Adjustment by john on 26/01/2010 17	26
39	gift_0001_30	Dotcomgiftshop Gift Voucher £30.00	17
38	gift_0001_20	Dotcomgiftshop Gift Voucher £20.00	17
31	PADS	PADS TO MATCH ALL CUSHIONS	15
35	TEST001	This is a test product.	15
24	DCGS0076	SUNJAR LED NIGHT NIGHT LIGHT	13
25	DCGSSBOY	BOYS PARTY BAG	10
27	DCGSSGIRL	GIRLS PARTY BAG	10
5	AMAZONFEE	AMAZON FEE	9
11	DCGS0003	BOXED GLASS ASHTRAY	9
37	gift_0001_10	Dotcomgiftshop Gift Voucher £10.00	6
7	BANK CHARGES	Bank Charges	6
44	m	Manual	4
20	DCGS0069	OOH LA LA DOGS COLLAR	4
1	ADJUST	Adjustment by Peter on 24/05/2010 1	3
6	B	Adjust bad debt	3
4	ADJUST2	Adjustment by Peter on Jun 25 2010	3
12	DCGS0004	HAYNES CAMPER SHOULDER BAG	3
22	DCGS0072	CAT CAMOUFLAGUE COLLAR	3
19	DCGS0068	DOGS NIGHT COLLAR	2
43	gift_0001_80	Dotcomgiftshop Gift Voucher £80.00	2

	StockCode	Description	Jumlah Muncul
18	DCGS0066N	NAVY CUDDLES DOG HOODIE	2
40	gift_0001_40	Dotcomgiftshop Gift Voucher £40.00	2
34	SP1002	KID'S CHALKBOARD/EASEL	2
41	gift_0001_50	Dotcomgiftshop Gift Voucher £50.00	2
0	47503J	SET/3 FLORAL GARDEN TOOLS IN BAG	1
13	DCGS0037	KEY-RING CORKSCREW	1
17	DCGS0062	ROAD-RAGE CAR FRESHENER	1
28	DCGSSGIRL	update	1
26	DCGSSBOY	update	1
21	DCGS0070	CAMOUFLAGE DOG COLLAR	1
23	DCGS0075	CAMOUFLAGUE DOG LEAD	1
15	DCGS0044	HANDZ-OFF CAR FRESHENER	1
14	DCGS0041	HAYNES MINI-COOPER PLAYING CARDS	1
36	TEST002	This is a test product.	1
42	gift_0001_70	Dotcomgiftshop Gift Voucher £70.00	1

```
In [71]: # Kodingan ini murni dari ai, saya hanya menyuruh dia untuk membuat kodingan agar b

# 1. Ambil lagi daftar kode aneh tadi
kode_aneh = df[(df["StockCode"].str.match("^\\d{5}$") == False) & (df["StockCode"].

# 2. Filter data cuma yang kode aneh
df_aneh = df[df["StockCode"].isin(kode_aneh)]

# 3. Kita Groupby untuk menghitung statistik Customer ID-nya
cek_customer = df_aneh.groupby("StockCode").agg(
    Total_Transaksi=('Invoice', 'count'),          # Hitung total baris
    Ada_CustomerID=('Customer ID', 'count'),        # Hitung yang ID-nya TIDAK NaN
    CustomerID_NaN=('Customer ID', lambda x: x.isna().sum()) # Hitung yang ID-nya N
).sort_values(by='Total_Transaksi', ascending=False)

# Tampilkan
cek_customer
```

Out[71]:

	Total_Transaksi	Ada_CustomerID	CustomerID_NaN
StockCode			
POST	865	822	43
M	850	650	200
DOT	736	0	736
C2	138	125	13
D	100	97	3
ADJUST	67	61	6
BANK CHARGES	65	26	39
S	41	0	41
DCGS0058	31	0	31
gift_0001_30	21	0	21
gift_0001_20	19	0	19
TEST001	15	15	0
PADS	15	15	0
DCGS0076	13	0	13
DCGSSBOY	12	0	12
DCGSSGIRL	12	0	12
AMAZONFEE	9	0	9
DCGS0003	9	0	9
gift_0001_10	7	0	7
DCGS0004	4	0	4
DCGS0069	4	0	4
DCGS0066N	4	0	4
gift_0001_40	4	0	4
gift_0001_50	4	0	4
gift_0001_80	4	0	4
m	4	0	4
DCGS0072	3	0	3
gift_0001_70	3	0	3
B	3	0	3

StockCode	Total_Transaksi	Ada_CustomerID	CustomerID_NaN
ADJUST2	3	3	0
SP1002	3	2	1
gift_0001_60	2	0	2
gift_0001_90	2	0	2
DCGS0068	2	0	2
DCGS0062	2	0	2
DCGS0037	2	0	2
TEST002	2	1	1
DCGS0016	1	0	1
47503J	1	0	1
DCGS0059	1	0	1
DCGS0056	1	0	1
DCGS0053	1	0	1
DCGS0044	1	0	1
DCGS0039	1	0	1
DCGS0041	1	0	1
DCGS0027	1	0	1
DCGS0036	1	0	1
C3	1	0	1
DCGS0006	1	0	1
GIFT	1	0	1
DCGSLGIRL	1	0	1
DCGS0070	1	0	1
DCGS0060	1	0	1
DCGSLBOY	1	0	1
DCGS0075	1	0	1

```
In [72]: # Daftar kode yang bikin kamu penasaran
kode_penasaran = ['POST', 'M', 'DOT', 'C2', 'D', 'ADJUST', 'BANK CHARGES', 'S', 'DC

# Loop untuk cek satu per satu
for kode in kode_penasaran:
```



```

print(f"=====")
print(f"👁 MENGINTIP ISI STOCK CODE: {kode}")
print(f"=====")

# Ambil data yang stockcode-nya sesuai
data_intip = df[df['StockCode'] == kode]

# Tampilkan 10 baris pertama saja biar gak penuh layarnya
display(data_intip.head(10))

# Kasih jarak enter biar enak bacanya
print("\n")

```

```

=====
👁 MENGINTIP ISI STOCK CODE: POST
=====

```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
89	489439	POST	POSTAGE	3	2009-12-01 09:28:00	18.00	12682.00	France
126	489444	POST	POSTAGE	1	2009-12-01 09:55:00	141.00	12636.00	USA
173	489447	POST	POSTAGE	1	2009-12-01 10:10:00	130.00	12362.00	Belgium
625	489526	POST	POSTAGE	6	2009-12-01 11:50:00	18.00	12533.00	Germany
927	C489538	POST	POSTAGE	-1	2009-12-01 12:18:00	9.58	15796.00	United Kingdom
1244	489557	POST	POSTAGE	4	2009-12-01 12:52:00	18.00	12490.00	France
3451	C489685	POST	POSTAGE	-1	2009-12-02 10:28:00	18.00	12523.00	France
6406	489883	POST	POSTAGE	3	2009-12-02 16:24:00	18.00	12437.00	France
9103	C490117	POST	POSTAGE	-1	2009-12-03 17:38:00	2.99	16570.00	United Kingdom
9153	C490120	POST	POSTAGE	-2	2009-12-03 17:52:00	18.00	14277.00	France

```

=====
👁 MENGINTIP ISI STOCK CODE: M
=====

```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
2697	489609	M	Manual	1	2009-12-01 14:50:00	4.00	NaN	United Kingdom
3053	C489651	M	Manual	-1	2009-12-01 16:48:00	5.10	17804.00	United Kingdom
5897	C489859	M	Manual	-1	2009-12-02 14:45:00	69.57	NaN	United Kingdom
9259	C490126	M	Manual	-1	2009-12-03 18:12:00	5.95	15884.00	United Kingdom
9307	C490129	M	Manual	-1	2009-12-03 18:26:00	1998.49	15482.00	United Kingdom
11310	490300	M	Manual	1	2009-12-04 14:19:00	0.85	12970.00	United Kingdom
11311	490300	M	Manual	1	2009-12-04 14:19:00	0.21	12970.00	United Kingdom
16107	490727	M	Manual	1	2009-12-07 16:38:00	0.00	17231.00	United Kingdom
17273	C490748	M	Manual	-1	2009-12-07 18:14:00	309.73	12748.00	United Kingdom
17386	490760	M	Manual	1	2009-12-08 09:49:00	10.00	14295.00	United Kingdom



```
=====
👁 MENGINTIP ISI STOCK CODE: DOT
=====
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
2379	489597	DOT	DOTCOM POSTAGE	1	2009-12-01 14:28:00	647.19	NaN	United Kingdom
2539	489600	DOT	DOTCOM POSTAGE	1	2009-12-01 14:43:00	55.96	NaN	United Kingdom
2551	489601	DOT	DOTCOM POSTAGE	1	2009-12-01 14:44:00	68.39	NaN	United Kingdom
2571	489602	DOT	DOTCOM POSTAGE	1	2009-12-01 14:45:00	59.35	NaN	United Kingdom
2619	489603	DOT	DOTCOM POSTAGE	1	2009-12-01 14:46:00	42.39	NaN	United Kingdom
2644	489604	DOT	DOTCOM POSTAGE	1	2009-12-01 14:47:00	50.87	NaN	United Kingdom
2686	489607	DOT	DOTCOM POSTAGE	1	2009-12-01 14:49:00	76.30	NaN	United Kingdom
2698	489609	DOT	DOTCOM POSTAGE	1	2009-12-01 14:50:00	74.61	NaN	United Kingdom
2771	489612	DOT	DOTCOM POSTAGE	1	2009-12-01 14:55:00	87.39	NaN	United Kingdom
2806	489614	DOT	DOTCOM POSTAGE	1	2009-12-01 14:56:00	80.30	NaN	United Kingdom

=====

👁 MENGINTIP ISI STOCK CODE: C2

=====

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
9292	490127	C2	CARRIAGE	1	2009-12-03 18:13:00	50.00	14156.00	EIRE
14481	490541	C2	CARRIAGE	1	2009-12-07 09:25:00	50.00	NaN	EIRE
14502	490542	C2	CARRIAGE	1	2009-12-07 09:42:00	50.00	14911.00	EIRE
19541	490998	C2	CARRIAGE	1	2009-12-08 17:24:00	50.00	16253.00	United Kingdom
22803	491160	C2	CARRIAGE	1	2009-12-10 10:29:00	50.00	14911.00	EIRE
27494	491702	C2	CARRIAGE	1	2009-12-13 13:53:00	50.00	NaN	EIRE
32207	491990	C2	NaN	100	2009-12-15 10:06:00	0.00	NaN	United Kingdom
32964	492092	C2	CARRIAGE	1	2009-12-15 14:03:00	50.00	14156.00	EIRE
34330	492250	C2	CARRIAGE	1	2009-12-16 10:45:00	50.00	18286.00	United Kingdom
39877	492746	C2	CARRIAGE	1	2009-12-18 13:01:00	50.00	NaN	EIRE

=====

👁 MENGINTIP ISI STOCK CODE: D

=====

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
735	C489535	D	Discount	-1	2009-12-01 12:11:00	9.00	15299.00	United Kingdom
736	C489535	D	Discount	-1	2009-12-01 12:11:00	19.00	15299.00	United Kingdom
24675	C491428	D	Discount	-1	2009-12-10 20:23:00	9.10	15494.00	United Kingdom
29414	C491845	D	Discount	-1	2009-12-14 14:54:00	1.59	NaN	United Kingdom
29958	C491962	D	Discount	-1	2009-12-14 16:38:00	0.59	13206.00	United Kingdom
39127	C492693	D	Discount	-1	2009-12-17 18:47:00	6.85	13408.00	United Kingdom
44782	C493373	D	Discount	-1	2009-12-23 11:22:00	64.37	15044.00	United Kingdom
62231	C494909	D	Discount	-30	2010-01-19 16:45:00	0.40	12931.00	United Kingdom
62232	C494909	D	Discount	-30	2010-01-19 16:45:00	0.13	12931.00	United Kingdom
62962	C494984	D	Discount	-1	2010-01-20 11:12:00	70.00	17949.00	United Kingdom

=====

👁 MENGINTIP ISI STOCK CODE: ADJUST

=====

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
70975	495732	ADJUST	Adjustment by john on 26/01/2010 16	1	2010-01-26 16:20:00	96.46	NaN	EIRE
70976	495733	ADJUST	Adjustment by john on 26/01/2010 16	1	2010-01-26 16:21:00	68.34	14911.00	EIRE
70977	495735	ADJUST	Adjustment by john on 26/01/2010 16	1	2010-01-26 16:22:00	201.56	12745.00	EIRE
70978	495734	ADJUST	Adjustment by john on 26/01/2010 16	1	2010-01-26 16:22:00	205.82	14911.00	EIRE
70979	C495737	ADJUST	Adjustment by john on 26/01/2010 16	-1	2010-01-26 16:23:00	10.50	16154.00	United Kingdom
70980	495736	ADJUST	Adjustment by john on 26/01/2010 16	1	2010-01-26 16:23:00	21.00	12606.00	Spain
70981	C495740	ADJUST	Adjustment by john on 26/01/2010 16	-1	2010-01-26 16:24:00	14.00	13054.00	United Kingdom
70982	C495738	ADJUST	Adjustment by john on 26/01/2010 16	-1	2010-01-26 16:24:00	26.25	12454.00	Spain
70983	C495739	ADJUST	Adjustment by john on 26/01/2010 16	-1	2010-01-26 16:24:00	10.50	15383.00	United Kingdom
70984	C495744	ADJUST	Adjustment by john on 26/01/2010 16	-1	2010-01-26 16:25:00	91.89	12706.00	Finland



=====

👤 MENGINTIP ISI STOCK CODE: BANK CHARGES

=====

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
18410	C490943	BANK CHARGES	Bank Charges	-1	2009-12-08 14:08:00	15.00	16703.00	United Kingdom
18466	490948	BANK CHARGES	Bank Charges	1	2009-12-08 14:29:00	15.00	16805.00	United Kingdom
33435	C492206	BANK CHARGES	Bank Charges	-1	2009-12-15 16:32:00	848.43	NaN	United Kingdom
55948	C494438	BANK CHARGES	Bank Charges	-1	2010-01-14 12:15:00	767.99	NaN	United Kingdom
94431	498269	BANK CHARGES	Bank Charges	1	2010-02-17 15:03:00	15.00	16928.00	United Kingdom
104220	C499374	BANK CHARGES	Bank Charges	-1	2010-02-26 11:55:00	467.54	NaN	United Kingdom
114180	C500319	BANK CHARGES	Bank Charges	-11	2010-03-07 12:02:00	0.96	NaN	United Kingdom
115208	C500352	BANK CHARGES	Bank Charges	-1	2010-03-07 15:08:00	11.29	NaN	United Kingdom
118558	C500708	BANK CHARGES	Bank Charges	-1	2010-03-09 13:56:00	372.30	NaN	United Kingdom
136411	C502459	BANK CHARGES	Bank Charges	-1	2010-03-24 14:25:00	39.24	NaN	United Kingdom



```
=====
👁 MENGINTIP ISI STOCK CODE: S
=====
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
114061	C500305	S	SAMPLES	-1	2010-03-07 10:59:00	73.80	NaN	United Kingdom
114083	C500309	S	SAMPLES	-1	2010-03-07 11:09:00	32.03	NaN	United Kingdom
133558	C502083	S	SAMPLES	-1	2010-03-22 15:50:00	170.37	NaN	United Kingdom
133582	C502088	S	SAMPLES	-1	2010-03-22 16:03:00	259.59	NaN	United Kingdom
136253	C502438	S	SAMPLES	-1	2010-03-24 13:11:00	605.18	NaN	United Kingdom
136259	C502442	S	SAMPLES	-1	2010-03-24 13:20:00	94.19	NaN	United Kingdom
181508	506601	S	SAMPLES	1	2010-04-30 14:49:00	73.80	NaN	United Kingdom
181509	C506602	S	SAMPLES	-1	2010-04-30 14:56:00	3.84	NaN	United Kingdom
181510	C506602	S	SAMPLES	-1	2010-04-30 14:56:00	3.55	NaN	United Kingdom
181511	C506602	S	SAMPLES	-1	2010-04-30 14:56:00	77.00	NaN	United Kingdom



```
=====
👁 MENGINTIP ISI STOCK CODE: DCGS0058
=====
```


	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
2377	489597	DCGS0058	MISO PRETTY GUM	1	2009-12-01 14:28:00	0.83	NaN	United Kingdom
8372	490074	DCGS0058	MISO PRETTY GUM	1	2009-12-03 14:39:00	0.83	NaN	United Kingdom
17264	490745	DCGS0058	MISO PRETTY GUM	1	2009-12-07 18:02:00	0.83	NaN	United Kingdom
30671	491969	DCGS0058	MISO PRETTY GUM	1	2009-12-14 17:57:00	0.83	NaN	United Kingdom
31652	491970	DCGS0058	MISO PRETTY GUM	1	2009-12-14 18:03:00	0.83	NaN	United Kingdom
32045	491971	DCGS0058	MISO PRETTY GUM	2	2009-12-14 18:37:00	0.83	NaN	United Kingdom
34668	492303	DCGS0058	MISO PRETTY GUM	1	2009-12-16 11:57:00	0.83	NaN	United Kingdom
37222	492425	DCGS0058	MISO PRETTY GUM	1	2009-12-16 17:58:00	0.83	NaN	United Kingdom
40878	492782	DCGS0058	MISO PRETTY GUM	1	2009-12-18 17:06:00	0.83	NaN	United Kingdom
41260	492783	DCGS0058	MISO PRETTY GUM	2	2009-12-18 17:15:00	0.83	NaN	United Kingdom

Stock Code

- StockCode is meant to follow the pattern `[0-9]{5}` but seems to have legit values for `[0-9]{5}[a-zA-Z]+`
 - Also contains other values:

Code	Description	Action
DCGS	Looks valid, some quantities are negative though and customer ID is null	Exclude from clustering

Code	Description	Action
D	Looks valid, represents discount values	Exclude from clustering
DOT	Looks valid, represents postage charges	Exclude from clustering
M or m	Looks valid, represents manual transactions	Exclude from clustering
C2	Carriage transaction - not sure what this means	Exclude from clustering
C3	Not sure, only 1 transaction	Exclude
BANK CHARGES or B	Bank charges	Exclude from clustering
S	Samples sent to customer	Exclude from clustering
TESTXXX	Testing data, not valid	Exclude from clustering
gift_XXX	Purchases with gift cards, might be interesting for another analysis, but no customer data	Exclude
PADS	Looks like a legit stock code for padding	Include
SP1002	Looks like a special request item, only 2 transactions, 3 look legit, 1 has 0 pricing	Exclude for now
AMAZONFEE	Looks like fees for Amazon shipping or something	Exclude for now
ADJUSTX	Looks like manual account adjustments by admins	Exclude for now

Kenapa kok bisa seperti itu? Pada dasar apa kita menghapus atau tetap memasukan suatu kategori?

Kembali ke tujuan utama kita pada sesi ini, yaitu : **Customer Segmentation**, kita ingin mengelompokkan orang berdasarkan "**Barang apa yang mereka suka beli**" dan ****"Seberapa loyal mereka belanja barang dagangan kita"**

Alasan lebih detailnya dibawah ini :

1. Kategori "Bukan Barang Dagangan" (Ini termasuk Jasa/Logistik)

- **Kode** Dot (Postage / Ongkos Kirim), C2 (Carriage), AMAZONFEE
- **Alasan Diubuang:**
 - Ini adalah biaya kirim atau biaya admin platform, yang berarti tidak ada kaitannya dengan apa yang ingin kita capai. Contoh: Kalau customer rumahnya jauh, akan menghabiskan lebih banyak uang dibandingkan dengan orang yang beli langsung. Sistem nantinya akan menganggap bahwa si customer yang alamatnya jauh ini lebih "Sultan" (Monetary tinggi). Padahal bisa saja dia beli

barangnya sedikit, sedangkan yang beli langsung beli barangnya cukup banyak namun kalah secara jumlah uang yang dikeluarkan karena biaya ongkir itu tadi.

2. Kategori Admin/Akuntansi

- Kode: **M** **m** (Manual), **D** (Discount), **BANK CHARGES**, **ADJUST**.
- **Alasan Diubuang:**
 - Untuk kode **M** **m** kita tidak bisa tau barang apa yang dibeli, sehingga akan bisa menjadi "sampah" bagi data latih kita. Begitu pula dengan **BANK CHARGES** yang merupakan pemberian otomatis dari Bank bersangkutan yang tidak memiliki arti apa-apa.

Sisanyaa sudah cukup intuitive untuk diketahui mengapa perlu dihapus. Pengecualian saja untuk

3. Kenapa **PADS** Dimasukkan (Include)?

- Kode: **PADS**
- Deskripsi: **PADS TO MATCH ALL CUSHIONS.**
- Alasan Disimpan:
 - Nah, ini pengecualian unik! Walaupun kodenya aneh (huruf semua), tapi kalau dilihat deskripsinya, ini adalah Barang Fisik (Busa/Bantalan Kursi). Ini adalah produk jualan beneran yang dipilih dan dibeli customer. Jadi ini **VALID** mencerminkan perilaku belanja.

Kesimpulan Strategi Si Trent (dan Kita):

Dia ingin membersihkan dataset supaya hanya menyisakan **TRANSAKSI PRODUK FISIK YANG VALID**.

Dengan begitu, nanti saat algoritma K-Means bekerja:

- Klaster 1 = Orang yang suka beli pernak-pernik kecil.
- Klaster 2 = Orang yang suka borong barang mahal.

Bukan jadi:

- Klaster 1 = Orang yang rumahnya jauh (Ongkir mahal).
- Klaster 2 = Orang yang sering kena denda Bank.

Data Cleaning

```
In [73]: cleaned_df = df.copy()
```

```
In [74]: cleaned_df["Invoice"] = cleaned_df["Invoice"].astype("str")

mask = (
```

```
cleaned_df["Invoice"].str.match("^\\d{6}$") == True
)

cleaned_df= cleaned_df[mask]
cleaned_df
```

Out[74]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13085.00	United Kingdom
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.00	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.00	United Kingdom
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13085.00	United Kingdom
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13085.00	United Kingdom
...	...	...	...	...	...	...	...	...
525456	538171	22271	FELTCRAFT DOLL ROSIE	2	2010-12-09 20:01:00	2.95	17530.00	United Kingdom
525457	538171	22750	FELTCRAFT PRINCESS LOLA DOLL	1	2010-12-09 20:01:00	3.75	17530.00	United Kingdom
525458	538171	22751	FELTCRAFT PRINCESS OLIVIA DOLL	1	2010-12-09 20:01:00	3.75	17530.00	United Kingdom
525459	538171	20970	PINK FLORAL FELTCRAFT SHOULDER BAG	2	2010-12-09 20:01:00	3.75	17530.00	United Kingdom
525460	538171	21931	JUMBO STORAGE BAG SUKI	2	2010-12-09 20:01:00	1.95	17530.00	United Kingdom

515252 rows × 8 columns



Kode ini berfungsi untuk membuang semua transaksi Aneh (termasuk "C" alias Cancellation dan "A" alias Bad Debt) dan hanya menyisakan transaksi Normal (Sesuai dengan deskripsi

dokumentasi).

```
In [75]: cleaned_df["StockCode"] = cleaned_df["StockCode"].astype("str")

mask = (
    (cleaned_df["StockCode"].str.match("^\\d{5}") == True)
    | (cleaned_df["StockCode"].str.match("^\\d{5}[a-zA-Z]+$") == True)
    | (cleaned_df["StockCode"].str.match("PADS") == True)
)

cleaned_df = cleaned_df[mask]
cleaned_df
```

Out[75]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13085.00	United Kingdom
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.00	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.00	United Kingdom
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13085.00	United Kingdom
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13085.00	United Kingdom
...	...	...	...	...	...	...	...	...
525456	538171	22271	FELTCRAFT DOLL ROSIE	2	2010-12-09 20:01:00	2.95	17530.00	United Kingdom
525457	538171	22750	FELTCRAFT PRINCESS LOLA DOLL	1	2010-12-09 20:01:00	3.75	17530.00	United Kingdom
525458	538171	22751	FELTCRAFT PRINCESS OLIVIA DOLL	1	2010-12-09 20:01:00	3.75	17530.00	United Kingdom
525459	538171	20970	PINK FLORAL FELTCRAFT SHOULDER BAG	2	2010-12-09 20:01:00	3.75	17530.00	United Kingdom
525460	538171	21931	JUMBO STORAGE BAG SUKI	2	2010-12-09 20:01:00	1.95	17530.00	United Kingdom

512797 rows × 8 columns

In [76]: `cleaned_df.describe()`

Out[76]:

	Quantity	InvoiceDate	Price	Customer ID
count	512797.00	512797	512797.00	406337.00
mean	11.00	2010-06-28 18:26:53.830657792	3.39	15373.63
min	-9600.00	2009-12-01 07:45:00	0.00	12346.00
25%	1.00	2010-03-21 13:27:00	1.25	14004.00
50%	3.00	2010-07-06 14:25:00	2.10	15326.00
75%	10.00	2010-10-15 14:50:00	4.21	16814.00
max	19152.00	2010-12-09 20:01:00	1157.15	18287.00
std	104.35	NaN	5.07	1677.37

Oke, ini adalah **Filter Tahap 2**, khusus untuk membersihkan kolom **StockCode** (Kode Barang).

Tujuannya: Membuang "sampah" seperti `POST`, `DOT`, `M`, `BANK CHARGES` yang tadi sudah kita diskusikan panjang lebar.

Mari kita bedah **3 Syarat Sakti** di dalam kode mask itu. Logikanya pakai **OR (|)**, artinya: "Kalau memenuhi SALAH SATU syarat ini, boleh masuk (disimpan)."

Bedah Syarat (Yang Disimpan)

- 1. `^\d{5}`** (Angka 5 Digit di depan)
 - **Maksud:** Kode barang standar/normal.
 - **Contoh Disimpan:** `85048`, `22041`.
 - *Note: Di regex ini dia cuma cek depannya 5 angka, belakangnya bebas. Tapi biasanya kode murni 5 angka masuk sini.*
- 2. `^\d{5}[a-zA-Z]+$`** (5 Angka + Huruf)
 - **Maksud:** Kode barang variasi warna/tipe.
 - **Contoh Disimpan:** `79323P` (Pink), `79323W` (White), `85123A`.
- 3. `PADS`** (Spesifik kata "PADS")
 - **Maksud:** Pengecualian unik yang tadi kita sepakati (Busa Kursi).
 - **Contoh Disimpan:** `PADS`.

Siapa yang Terbuang? (Contoh yang Dihapus)

Semua kode yang **TIDAK** memenuhi 3 syarat di atas akan dihapus.

- `POST` (Postage) -> Dibuang (Karena huruf semua, bukan PADS).

- **DOT** (Dotcom Postage) -> Dibuang.
- **M** (Manual) -> Dibuang.
- **C2** (Carriage) -> Dibuang (Karena depannya huruf C, bukan angka).
- **BANK CHARGES** -> Dibuang.

Ilustrasi Before vs After

Sebelum Filter (`cleaned_df` awal):

Invoice	StockCode	Description	Status
536365	85123A	White Heart...	✓ (Masuk Syarat 2)
536365	71053	White Metal...	✓ (Masuk Syarat 1)
536365	POST	Postage	✗ (Gak lolos semua syarat)
536365	PADS	Pads to match...	✓ (Masuk Syarat 3 - Spesial)
536365	M	Manual	✗ (Gak lolos)

Sesudah Filter (`cleaned_df` akhir):

Invoice	StockCode	Description
536365	85123A	White Heart...
536365	71053	White Metal...
536365	PADS	Pads to match...

(Lihat? **POST** dan **M** sudah hilang. Data jadi bersih murni barang fisik).

Jujur menurut saya, apa yang dilakukan oleh Trent (Tutorial) secara pemrograman, syarat 1 dan 2 sebenarnya sama aja karena

Syarat 1: `^\d{5}` (Tanpa Dolar)

- Artinya: "Pokoknya **DEPANNYA** harus 5 angka. Belakangnya terserah mau ada apa kek, mau kosong kek, mau huruf kek."
- Efeknya: Ini sebenarnya **SUDAH MENCAKUP** Syarat 2.
 - `12345` -> Lolos (Depannya 5 angka).
 - `12345A` -> Lolos (Depannya 5 angka).
 - `12345Abc` -> Lolos (Depannya 5 angka).
- **Kelemahan:** Kalau ada kode `12345!@#`, dia juga bakal lolos (kalau regexnya longgar begini).

Syarat 2: `^\d{5}[a-zA-Z]+$`

- Artinya: "Harus 5 angka, **LALU WAJIB** diikuti huruf sampai habis."

- Ini lebih spesifik/ketat.

Kenapa si Trent (Tutorial) nulis dua-duanya?

Jujur, secara logika pemrograman, kalau Syarat 1 ditulisnya longgar (`^\d{5}` tanpa `$` di akhir), maka Syarat 2 itu sebenarnya **redundant (mubazir)** karena `12345A` sudah pasti ketangkap sama Syarat 1.

TAPI... Biasanya ini dilakukan untuk **Kejelasan (Readability)** atau jaga-jaga (Defensive Programming).

- Dia ingin memastikan secara eksplisit: "Saya mau yang angka doang (Syarat 1 biasanya diasumsikan angka tok)" DAN "Saya mau yang ada hurufnya (Syarat 2)".

```
In [77]: cleaned_df.dropna(subset=["Customer ID"], inplace=True)
```

C:\Users\MSI KATANA 15\AppData\Local\Temp\ipykernel_18036\1633333693.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
cleaned_df.dropna(subset=["Customer ID"], inplace=True)

```
In [78]: print("--- BEFORE ---")
df.describe()
```

--- BEFORE ---

```
Out[78]:
```

	Quantity	InvoiceDate	Price	Customer ID
count	525461.00	525461	525461.00	417534.00
mean	10.34	2010-06-28 11:37:36.845017856	4.69	15360.65
min	-9600.00	2009-12-01 07:45:00	-53594.36	12346.00
25%	1.00	2010-03-21 12:20:00	1.25	13983.00
50%	3.00	2010-07-06 09:51:00	2.10	15311.00
75%	10.00	2010-10-15 12:45:00	4.21	16799.00
max	19152.00	2010-12-09 20:01:00	25111.09	18287.00
std	107.42	NaN	146.13	1680.81

```
In [79]: print("--- AFTER ---")
cleaned_df.describe()
```

--- AFTER ---

Out[79]:

	Quantity	InvoiceDate	Price	Customer ID
count	406337.00	406337	406337.00	406337.00
mean	13.62	2010-07-01 10:11:06.543288320	2.99	15373.63
min	1.00	2009-12-01 07:45:00	0.00	12346.00
25%	2.00	2010-03-26 14:01:00	1.25	14004.00
50%	5.00	2010-07-09 15:48:00	1.95	15326.00
75%	12.00	2010-10-14 17:09:00	3.75	16814.00
max	19152.00	2010-12-09 20:01:00	295.00	18287.00
std	97.00	NaN	4.29	1677.37

Dapat dilihat untuk nilai negatif pada **Quantity** sudah tidak ada dan nilai sekarang sudah sesuai ekspektasi. Namun masih ada yang janggal yaitu kolom **Price** yang nominalnya adalah 0. Barang dengan harga apa yang nominalnya adalah 0? Apakah mungkin karena ada promo, beli 1 gratis 1? beli barang A dapat barang B?

Mari kita investigasi!

In [80]: `cleaned_df[cleaned_df["Price"]== 0].head(5)`

Out[80]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
4674	489825	22076	6 RIBBONS EMPIRE	12	2009-12-02 13:34:00	0.00	16126.00	United Kingdom
6781	489998	48185	DOOR MAT FAIRY CAKE	2	2009-12-03 11:19:00	0.00	15658.00	United Kingdom
18738	490961	22065	CHRISTMAS PUDDING TRINKET POT	1	2009-12-08 15:25:00	0.00	14108.00	United Kingdom
18739	490961	22142	CHRISTMAS CRAFT WHITE FAIRY	12	2009-12-08 15:25:00	0.00	14108.00	United Kingdom
32916	492079	85042	ANTIQUE LILY FAIRY LIGHTS	8	2009-12-15 13:49:00	0.00	15070.00	United Kingdom

In [81]: `len(cleaned_df[cleaned_df["Price"]== 0])`

Out[81]: 28

```
In [82]: cleaned_df = cleaned_df[cleaned_df["Price"] > 0 ]
```

```
In [83]: cleaned_df.describe()
```

```
Out[83]:
```

	Quantity	InvoiceDate	Price	Customer ID
count	406309.00	406309	406309.00	406309.00
mean	13.62	2010-07-01 10:14:25.869572352	2.99	15373.72
min	1.00	2009-12-01 07:45:00	0.00	12346.00
25%	2.00	2010-03-26 14:01:00	1.25	14006.00
50%	5.00	2010-07-09 15:48:00	1.95	15326.00
75%	12.00	2010-10-14 17:09:00	3.75	16814.00
max	19152.00	2010-12-09 20:01:00	295.00	18287.00
std	97.00	NaN	4.29	1677.33

hmm masih 0 (kolom `Price`), apakah harganya sangat amat kecil mendekati nol?

```
In [84]: cleaned_df["Price"].min()
```

```
Out[84]: np.float64(0.001)
```

Untuk sekarang, sepertinyaa tidak apa-apa untuk kita include kan, karena yang penting harga == 0 sudah terhapus

```
In [85]: len(cleaned_df)/len(df) # Good Practise untuk mengetahui berapa banyak data yang hil
```

```
Out[85]: 0.7732429238325965
```

Data yang hilang sebanyak sekitar 23%

```
In [86]: # Simpan data yang sudah bersih ke file CSV baru
# index=False biar nomor baris (0, 1, 2...) gak ikut kesimpn jadi kolom baru
cleaned_df.to_csv("../data/online_retail_clean.csv", index=False)
```

How Does KMeans Clustering Work?

Sangat direkomendasikan untuk menonton video ini:



Theoretical Background: K-Means Clustering

1. Definisi K-Means

K-Means adalah algoritma **Unsupervised Learning** yang bertujuan untuk mengelompokkan data ke dalam K klaster (kelompok) yang berbeda. Algoritma ini bekerja secara iteratif untuk mempartisi data sedemikian rupa sehingga:

1. Setiap titik data hanya masuk ke dalam satu klaster.
2. Data dalam satu klaster memiliki kemiripan sifat (jarak yang dekat).
3. Data antar klaster memiliki perbedaan yang signifikan (jarak yang jauh).

2. Cara Kerja Algoritma (Step-by-Step)

Berdasarkan penjelasan StatQuest, alur kerja K-Means adalah sebagai berikut:

1. **Initialization (Inisialisasi):** Tentukan jumlah klaster (K) yang diinginkan. Algoritma akan memilih K titik acak sebagai pusat klaster awal (**Centroids**).
2. **Assignment (Pengelompokan):** Hitung jarak setiap titik data ke setiap Centroid (biasanya menggunakan *Euclidean Distance*). Masukkan titik data ke klaster dengan Centroid terdekat.

3. **Update Centroid (Pembaruan Pusat):** Hitung rata-rata (**Mean**) dari posisi semua titik data dalam satu kluster. Geser posisi Centroid ke titik rata-rata tersebut.
 4. **Iteration (Pengulangan):** Ulangi langkah 2 dan 3. Posisi Centroid akan terus bergeser dan anggota kluster mungkin berubah.
 5. **Convergence (Konvergensi):** Algoritma berhenti ketika posisi Centroid tidak lagi berubah (stabil), atau perubahan variasi sudah sangat tidak signifikan.
-

3. Key Metric: Variation (Inertia)

Dalam K-Means, kita tidak memiliki label "Benar" atau "Salah" (karena Unsupervised). Oleh karena itu, kita menggunakan **Variation** (dalam Scikit-Learn disebut **Inertia**) untuk mengukur kualitas kluster.

Apa itu Variasi/Inertia?

Variasi adalah ukuran seberapa "padat" atau "rapat" data di dalam sebuah kluster.

- Secara matematis, ini adalah **Within-Cluster Sum of Squares (WCSS)**.
- Rumusnya adalah penjumlahan dari jarak kuadrat antara setiap titik data (x) dengan pusat klusternya (C).

$$\text{Inertia} = \sum_{i=1}^N (x_i - C_k)^2$$

Intuisi Sederhana:

- **Variasi Besar:** Data tersebar jauh dari pusatnya (Kluster terlalu luas/berantakan).
 - **Variasi Kecil:** Data berkumpul rapat di dekat pusatnya (Kluster padat/kompak).
 - **Tujuan K-Means:** Meminimalisir nilai Variasi ini sekecil mungkin.
-

4. Menentukan K Terbaik: The Elbow Method

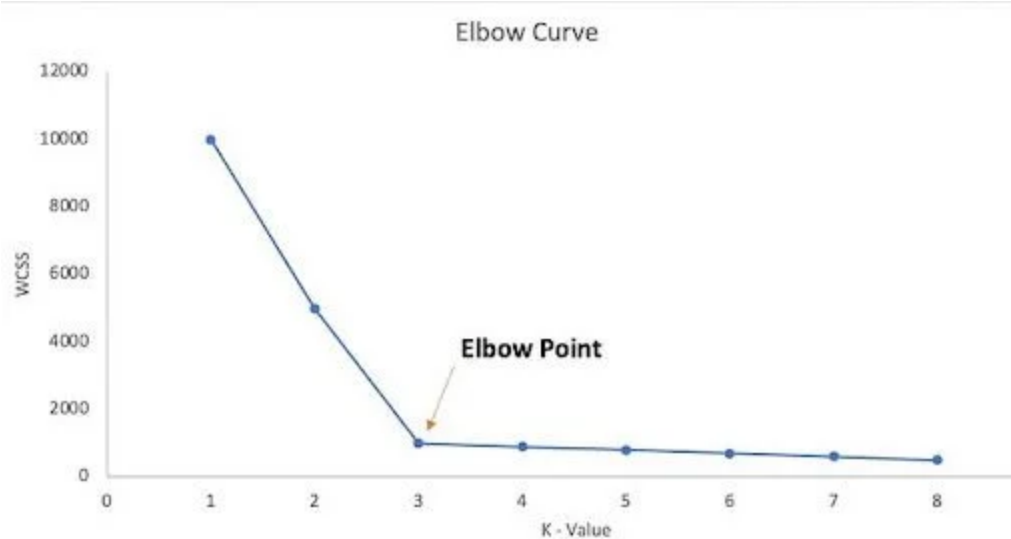
Salah satu tantangan K-Means adalah kita harus menentukan jumlah K di awal.

- Jika $K = 1$: Variasi sangat besar (Error tinggi).
- Jika $K = \text{Jumlah Data}$: Variasi = 0 (Setiap data adalah klusternya sendiri), namun ini tidak berguna untuk segmentasi.

Elbow Method digunakan untuk mencari keseimbangan:

1. Plot nilai **Inertia (Y-axis)** terhadap jumlah **Kluster K (X-axis)**.
2. Grafik akan menurun drastis di awal, lalu melandai.

3. Titik sudut di mana penurunan mulai melandai (membentuk siku/elbow) adalah jumlah K yang optimal.
- Artinya: Menambah kluster lagi setelah titik ini tidak memberikan penurunan variasi yang signifikan (*Diminishing Returns*).



Biasanya, memang ada debat antara ingin menggunakan titik ke tiga atau ke empat. Seringkalinya ditentukan pada konteks bisnis atau tujuan akhir yang ingin kita capai

- Kalau kamu punya 100 data, dan kamu bikin $K = 100$ (setiap orang punya kelompok sendiri-sendiri), maka **Variance-nya = 0**. (Jarak ke dirinya sendiri kan nol).

Terus kalau Variance 0 itu bagus secara matematika, **kenapa kita nggak pakai $K=100$ aja sekalian?**

Jawabannya: **Karena TIDAK BERGUNA secara Bisnis.**

Bayangkan Bos kamu minta segmentasi customer.

- **Kamu:** "Bos, saya sudah bikin segmentasi paling sempurna (Variance 0). Ada 100.000 kelompok untuk 100.000 customer kita."
- **Bos:** "Lah? Terus tim marketing saya harus bikin 100.000 strategi promosi beda-beda gitu? Pecat aja saya sekalian!"

Di sinilah Seni "Elbow Method" Masuk (Trade-off)

Kita mencari titik "**Keseimbangan**" (**Sweet Spot**). Antara:

1. **Akurasi (Variance Kecil)**
2. **Kesederhanaan (Jumlah K Sedikit)**

Mari kita lihat grafik Siku (Elbow) lagi:

- Dari $K = 1$ ke $K = 2$: Variance turun **DRASTIS** (Misal dari 1000 $\rightarrow$ 400).
 - *Artinya*: Menambah 1 kelompok itu untung banget! Informasinya jadi jauh lebih jelas.
- Dari $K = 2$ ke $K = 3$: Variance turun lumayan (Misal 400 $\rightarrow$ 200).
 - *Artinya*: Masih untung.
- Dari $K = 3$ ke $K = 4$: Variance turun dikiiit banget (Misal 200 $\rightarrow$ 180).
 - *Artinya*: Nah, di sini mulai rugi. Kamu nambah kerjaan (nambah kelompok), tapi perbaikan datanya cuma seuprit. Ini namanya *Diminishing Returns*.

Titik Siku (Elbow) adalah momen di mana "Keuntungan" (penurunan variance) mulai kalah sama "Keribetan" (nambah kelompok).

Soal Debat K=3 vs K=4 (Konteks Bisnis)

Seringkali secara matematika sikunya ada di antara 3 dan 4. Pilih yang mana? Di sinilah peran kamu sebagai **Data Analyst** (bukan cuma tukang ngitung). Kamu harus tanya ke Tim Bisnis/Marketing:

Skenario Debat:

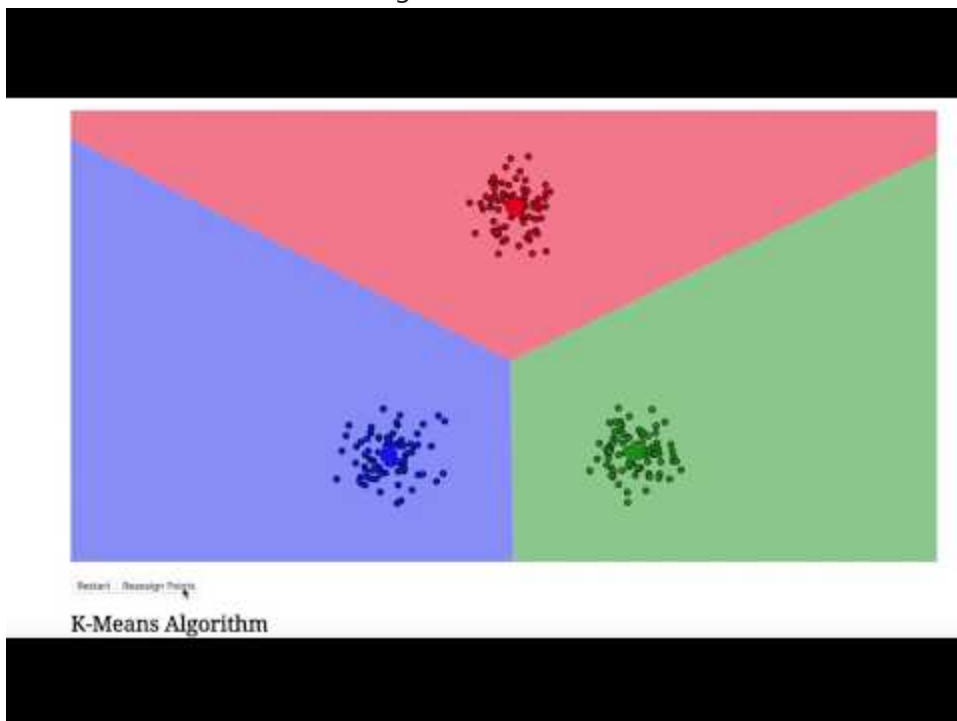
- **Matematika**: $K=3$ variance-nya 200, $K=4$ variance-nya 180. (Beda tipis).
- **Analisis Bisnis**:
 - **Kalau $K=3$** : Ada kelompok "Low", "Mid", "High". (Simpel, gampang dieksekusi marketing).
 - **Kalau $K=4$** : Ada kelompok "Low", "Mid", "High", dan "Very High".

Pertanyaannya: "Apakah tim marketing punya budget/tenaga buat bikin perlakuan khusus ke si 'Very High' ini?"

- Kalau marketing bilang: "*Ah, budget kita terbatas, yang penting pisahin aja yang kaya sama yang miskin.*" $\rightarrow$ **Pilih $K=3$** .
- Kalau marketing bilang: "*Kita mau kasih hadiah Emas buat top 1% sultan!*" $\rightarrow$ **Pilih $K=4$** (supaya si "Very High" terpisah dari "High").

Kesimpulan: Variance harus kecil, TAPI jumlah K harus **Actionable** (Bisa dieksekusi). Jangan mengejar Variance 0, tapi kejarlah Variance yang "Cukup Rendah" dengan jumlah kelompok yang "Masuk Akal".

For more intuitive understanding:



or you can play with this : [Visualizing K-Means Clustering - Naftali Harris](#)

3. Feature Engineering: Metodologi RFM

Untuk mengubah data transaksi mentah menjadi profil pelanggan yang dapat ditindaklanjuti, saya mengikuti video tutorial, si Trent menerapkan kerangka kerja analisis **RFM (Recency, Frequency, Monetary)**. Metode ini merupakan standar industri dalam ritel dan pemasaran untuk mengevaluasi nilai dan retensi pelanggan.

Tujuan dari tahap ini adalah melakukan agregasi data dari tingkat transaksi (banyak baris per pelanggan) menjadi data tingkat pelanggan (satu baris per pelanggan), yang dikarakterisasi oleh tiga metrik kuantitatif berikut:

1. Recency (R) - Kebaruan

- **Definisi:** Mengukur jumlah hari yang berlalu sejak transaksi terakhir pelanggan dibandingkan dengan tanggal referensi (tanggal snapshot data).
- **Signifikansi Bisnis:** Mengindikasikan tingkat keterlibatan (*engagement*) pelanggan. Nilai recency yang rendah menunjukkan keterlibatan tinggi, sedangkan nilai yang tinggi mengindikasikan risiko pelanggan berhenti berlangganan (*churn*).
- **Perhitungan:** Tanggal Referensi - Tanggal Faktur Terakhir

2. Frequency (F) - Frekuensi

- **Definisi:** Jumlah total transaksi unik (faktur) yang berhasil dilakukan oleh pelanggan dalam periode analisis.
- **Signifikansi Bisnis:** Mencerminkan loyalitas dan kebiasaan pembelian pelanggan. Metrik ini membedakan antara pembeli sekali (*one-time buyer*) dengan pelanggan tetap.
- **Perhitungan:** Menghitung jumlah unik `Invoice` per pelanggan.

3. Monetary (M) - Moneter

- **Definisi:** Total nilai uang (pendapatan) yang dikontribusikan oleh pelanggan kepada perusahaan.
- **Signifikansi Bisnis:** Mencerminkan daya beli dan profitabilitas pelanggan. Hal ini membantu dalam mengidentifikasi pelanggan bernilai tinggi (*High-Value Customers*) yang memberikan dampak signifikan terhadap omzet.
- **Perhitungan:** Penjumlahan dari (`Quantity` * `Price`) untuk seluruh transaksi.

Ilustrasi Transformasi Data

Dengan menerapkan metodologi ini, struktur data akan bertransformasi sebagai berikut:

Input: Data Transaksi Mentah

Invoice	Customer ID	Tanggal	Quantity	Price
536365	17850	01-12-2010	6	2.55
536365	17850	01-12-2010	8	3.39
536366	17850	02-12-2010	6	2.55

Output: Fitur RFM Tingkat Pelanggan

Customer ID	Recency (Hari)	Frequency (Kali)	Monetary (\$)
17850	1	2	41.34

Catatan: Pada contoh di atas, Pelanggan 17850 memiliki 2 faktur unik (Frequency), terakhir berbelanja 1 hari yang lalu (Recency), dengan total belanja yang diakumulasikan dari seluruh barang (Monetary).

```
In [87]: cleaned_df["SalesLineTotal"] = cleaned_df["Quantity"] * cleaned_df["Price"]
cleaned_df
```

C:\Users\MSI KATANA 15\AppData\Local\Temp\ipykernel_18036\2846558921.py:1: SettingWithCopyWarning:
 A value is trying to be set on a copy of a slice from a DataFrame.
 Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
cleaned_df["SalesLineTotal"] = cleaned_df["Quantity"] * cleaned_df["Price"]
```

Out[87]:

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13085.00	United Kingdom
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.00	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.00	United Kingdom
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13085.00	United Kingdom
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13085.00	United Kingdom
...	...	...	...	...	...	...	...	...
525456	538171	22271	FELTCRAFT DOLL ROSIE	2	2010-12-09 20:01:00	2.95	17530.00	United Kingdom
525457	538171	22750	FELTCRAFT PRINCESS LOLA DOLL	1	2010-12-09 20:01:00	3.75	17530.00	United Kingdom
525458	538171	22751	FELTCRAFT PRINCESS OLIVIA DOLL	1	2010-12-09 20:01:00	3.75	17530.00	United Kingdom
525459	538171	20970	PINK FLORAL FELTCRAFT SHOULDER BAG	2	2010-12-09 20:01:00	3.75	17530.00	United Kingdom
525460	538171	21931	JUMBO STORAGE BAG SUKI	2	2010-12-09 20:01:00	1.95	17530.00	United Kingdom

406309 rows × 9 columns

```
In [88]: aggregated_df = cleaned_df.groupby(by="Customer ID", as_index=False) \
        .agg(
            MonetaryValue = ("SalesLineTotal", "sum"),
            Frequency = ("Invoice", "nunique"),
            LastInvoiceDate = ("InvoiceDate", "max")
        )
        aggregated_df.head(10)
```

```
Out[88]:
```

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate
0	12346.00	169.36	2	2010-06-28 13:53:00
1	12347.00	1323.32	2	2010-12-07 14:57:00
2	12348.00	221.16	1	2010-09-27 14:59:00
3	12349.00	2221.14	2	2010-10-28 08:23:00
4	12351.00	300.93	1	2010-11-29 15:23:00
5	12352.00	343.80	2	2010-11-29 10:07:00
6	12353.00	317.76	1	2010-10-27 12:44:00
7	12355.00	488.21	1	2010-05-21 11:59:00
8	12356.00	3126.25	3	2010-11-24 12:24:00
9	12357.00	11229.99	1	2010-11-16 10:05:00

Saya penasaran bagaimana logika di belakang layar itu terjadi, saya pengen tau setiap langkah itu apa sih yang terjadi

Bayangkan kita punya potongan data kecil ini (fokus pada 2 orang saja: ID **13085** dan ID **17530**):

Invoice	StockCode	...	Quantity	Price	Customer ID	SalesLineTotal	InvoiceDate
489434	85048	...	12	6.95	13085	83.40	2009-12-01
489434	79323P	...	12	6.75	13085	81.00	2009-12-01
538171	22271	...	2	2.95	17530	5.90	2010-12-09
538171	22750	...	1	3.75	17530	3.75	2010-12-09
538171	22751	...	1	3.75	17530	3.75	2010-12-09

Step 1: groupby(by="Customer ID")

Pandas akan memecah tabel besar itu menjadi beberapa "**Keranjang Kecil**" berdasarkan Customer ID.

Keranjang 1: Customer 13085

- Isinya 2 baris (Transaksi Invoice 489434 semua).
- Data: SalesLineTotal [83.40, 81.00], Invoice [489434, 489434], Date [2009-12-01, 2009-12-01].

Keranjang 2: Customer 17530

- Isinya 3 baris (Transaksi Invoice 538171 semua).
- Data: SalesLineTotal [5.90, 3.75, 3.75], Invoice [538171, 538171, 538171], Date [2010-12-09...].

Step 2: `.agg(...)` (Menjalankan Operasi Matematika di Tiap Keranjang)

Sekarang Pandas masuk ke tiap keranjang dan menghitung rumus yang kamu minta.

A. Hitung `MonetaryValue` ("SalesLineTotal", "sum")

- **Keranjang 13085:**
 - Ambil data SalesLineTotal: [83.40, 81.00]
 - Jumlahkan: $83.40 + 81.00 = 164.40$
- **Keranjang 17530:**
 - Ambil data SalesLineTotal: [5.90, 3.75, 3.75]
 - Jumlahkan: $5.90 + 3.75 + 3.75 = 13.40$

B. Hitung `Frequency` ("Invoice", "nunique")

- **Keranjang 13085:**
 - Ambil data Invoice: [489434, 489434]
 - Cari Unik: Cuma ada satu jenis angka yaitu 489434 .
 - Hitung Jumlah Unik: 1
- **Keranjang 17530:**
 - Ambil data Invoice: [538171, 538171, 538171]
 - Cari Unik: Cuma ada satu jenis angka yaitu 538171 .
 - Hitung Jumlah Unik: 1

(Catatan: Kalau misal Customer 13085 besok belanja lagi dengan Invoice baru 999999, maka listnya jadi [489434, 489434, 999999]. Jumlah Uniknya jadi 2).

C. Hitung `LastInvoiceDate` ("InvoiceDate", "max")

- **Keranjang 13085:**
 - Ambil data Date: [2009-12-01, 2009-12-01]
 - Cari Maksimum: 2009-12-01

- **Keranjang 17530:**
 - Ambil data Date: [2010-12-09, 2010-12-09, 2010-12-09]
 - Cari Maksimum: **2010-12-09**

Step 3: Penggabungan Kembali (Final Output)

Setelah hitung-hitungan selesai di tiap keranjang, Pandas menyatukan hasilnya jadi tabel baru (`aggregated_df`):

Customer ID	MonetaryValue	Frequency	LastInvoiceDate
13085	164.40	1	2009-12-01
17530	13.40	1	2010-12-09

Ini lah yang kamu lihat di output `head(10)` . Satu baris sudah mewakili satu orang dengan rangkuman perilaku belanjanya.

In [89]: `aggregated_df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4285 entries, 0 to 4284
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Customer ID     4285 non-null   float64
1   MonetaryValue   4285 non-null   float64
2   Frequency       4285 non-null   int64
3   LastInvoiceDate 4285 non-null   datetime64[ns]
dtypes: datetime64[ns](1), float64(2), int64(1)
memory usage: 134.0 KB
```

Dari 400.000 data, sekarang tinggal 4285 data, Artinya, selama periode 2009-2010, toko ini melayani 4.285 orang berbeda (yang datanya valid/lengkap).

```
In [90]: max_invoice_date = aggregated_df["LastInvoiceDate"].max()

aggregated_df["Recency"] = (max_invoice_date - aggregated_df["LastInvoiceDate"]).dt

aggregated_df.head(10)
```

Out[90]:

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	Recency
0	12346.00	169.36	2	2010-06-28 13:53:00	164
1	12347.00	1323.32	2	2010-12-07 14:57:00	2
2	12348.00	221.16	1	2010-09-27 14:59:00	73
3	12349.00	2221.14	2	2010-10-28 08:23:00	42
4	12351.00	300.93	1	2010-11-29 15:23:00	10
5	12352.00	343.80	2	2010-11-29 10:07:00	10
6	12353.00	317.76	1	2010-10-27 12:44:00	43
7	12355.00	488.21	1	2010-05-21 11:59:00	202
8	12356.00	3126.25	3	2010-11-24 12:24:00	15
9	12357.00	11229.99	1	2010-11-16 10:05:00	23



Catatan Perhitungan Recency

Dalam skenario bisnis *real-time*, metrik **Recency** umumnya dihitung berdasarkan tanggal saat ini (*today*) menggunakan fungsi waktu sistem.

Namun, karena analisis ini menggunakan **dataset historis** (transaksi tahun 2009-2010), penggunaan tanggal saat ini (2026) tidak relevan karena akan menghasilkan nilai Recency yang ekstrem (>5000 hari) untuk seluruh pelanggan.

Metodologi: Kami menetapkan titik referensi waktu berdasarkan **tanggal transaksi terakhir** yang tercatat dalam dataset.

- **Formula:** $\text{Recency} = \text{Max}(\text{InvoiceDate}) - \text{LastInvoiceDate}$
- **Interpretasi:** Jumlah hari yang telah berlalu sejak pembelian terakhir pelanggan relatif terhadap akhir periode data.

```
In [91]: plt.figure(figsize=(15,5))

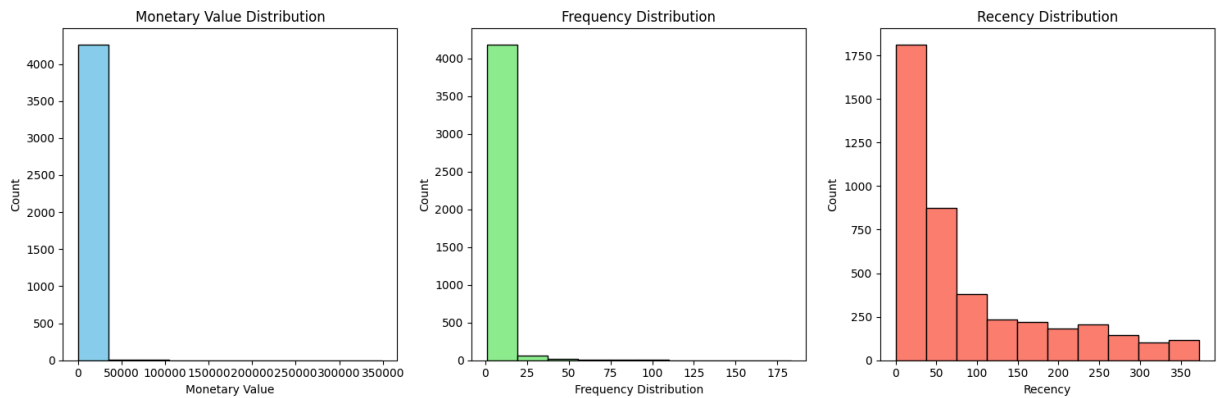
plt.subplot(1,3,1)
plt.hist(aggregated_df['MonetaryValue'], bins=10, color='skyblue', edgecolor='black')
plt.title('Monetary Value Distribution')
plt.xlabel('Monetary Value')
plt.ylabel('Count')

plt.subplot(1,3,2)
plt.hist(aggregated_df['Frequency'], bins=10, color='lightgreen', edgecolor='black')
plt.title('Frequency Distribution')
plt.xlabel('Frequency Distribution')
plt.ylabel('Count')

plt.subplot(1,3,3)
```

```
plt.hist(aggregated_df['Recency'], bins=10, color='salmon', edgecolor='black')
plt.title('Recency Distribution')
plt.xlabel('Recency')
plt.ylabel('Count')

plt.tight_layout()
plt.show()
```



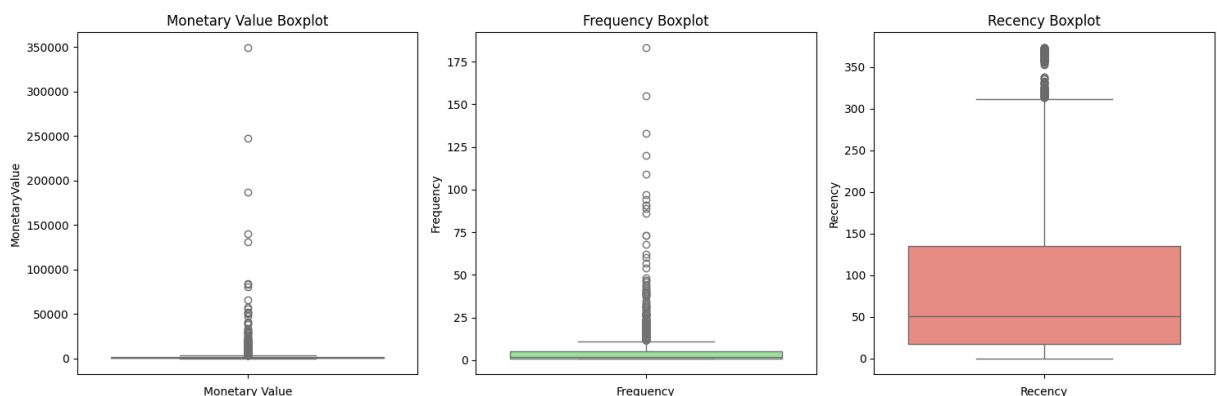
```
In [92]: # Metode paling cocok buat ngecek Outliers
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
sns.boxplot(data=aggregated_df['MonetaryValue'], color='skyblue')
plt.title('Monetary Value Boxplot')
plt.xlabel('Monetary Value')

plt.subplot(1, 3, 2)
sns.boxplot(data=aggregated_df['Frequency'], color='lightgreen')
plt.title('Frequency Boxplot')
plt.xlabel('Frequency')

plt.subplot(1, 3, 3)
sns.boxplot(data=aggregated_df['Recency'], color='salmon')
plt.title('Recency Boxplot')
plt.xlabel('Recency')

plt.tight_layout()
plt.show()
```



Kuncinya:

- **Tabel Distribusi** (Lihat bawah pas line kode ini) melihat "**Berapa banyak ORANG di rentang nilai tertentu**".
 - **Boxplot** melihat "**Di nilai BERAPA sih posisi orang ke-25%, ke-50%, dan ke-75%?**".
-

Cara Baca Boxplot (Versi Detail)

Kita fokus ke **Recency (Kotak Merah)** karena paling jelas bentuknya.

Bayangkan kamu punya 100 orang berbarisurut dari yang paling rajin belanja (Recency 0) sampai yang paling malas (Recency 373).

1. Garis Bawah Kotak (Q1 / 25%):

- Lihat garis bawah kotak merah itu ada di angka sekitar **17**.
- *Artinya:* Orang urutan ke-25 (seperempat barisan terdepan) punya nilai Recency 17.
- *Terjemahan:* "25% pelanggan ter-rajin kita, belanja terakhirnya kurang dari 17 hari yang lalu."

2. Garis Tengah Kotak (Median / 50%):

- Garis yang membelah kotak ada di angka sekitar **50**.
- *Artinya:* Orang urutan ke-50 (tengah-tengah barisan) punya nilai Recency 50.
- *Terjemahan:* "Separuh pelanggan kita (50%) belanja terakhirnya kurang dari 50 hari yang lalu."

3. Garis Atas Kotak (Q3 / 75%):

- Garis tutup kotak merah ada di angka sekitar **140**.
 - *Artinya:* Orang urutan ke-75 punya nilai Recency 140.
 - *Terjemahan:* "75% pelanggan kita (sebagian besar) belanja terakhirnya kurang dari 140 hari yang lalu."
-

Bandingkan dengan Tabel Distribusi (Recency)

Tabel Teks tadi bilang:

- Rentang **0 - 37 Hari = 42.33% Orang**.

Cek ke Boxplot:

- Garis tengah (Median 50%) ada di angka 50-an hari.
 - *Cocok kan?* Di tabel bilang 42% orang ada di bawah 37 hari. Di boxplot bilang 50% orang ada di bawah 50 hari. Angkanya nyambung/konsisten.
-

Kenapa Monetary (Biru) Kotaknya Gepeng?

Cek Tabel Teks Monetary:

- Rentang **0 - 34.000 = 99.53% Orang**.

Artinya:

- Orang ke-25% (Q1) → Belanjanya dikit (Misal 200).
- Orang ke-50% (Median) → Belanjanya dikit (Misal 500).
- Orang ke-75% (Q3) → Belanjanya dikit (Misal 1000).

Karena angka 200, 500, dan 1000 itu **SANGAT KECIL** dibandingkan Sultan yang belanja **350.000**, maka di gambar Boxplot:

- Garis Q1, Median, Q3 semuanya **Gencet-gencetan** di bawah (dekat angka 0).
- Makanya kotaknya jadi gepeng kayak garis doang.
- Titik-titik di atasnya itu si Sultan (Outlier) yang bikin skala Y-nya jadi melar ke atas.

```
In [93]: # Disini saya meminta tolong kepada AI untuk "ah i see, terus ini bisa gak yaa gamb

# Fungsi untuk bikin Tabel Distribusi Frekuensi
def buat_tabel_distribusi(kolom, nama_kolom, n_bins=10):
    # 1. Bikin rentang (binning)
    # pd.cut memotong data jadi 10 bagian
    counts = pd.cut(aggregated_df[kolom], bins=n_bins).value_counts().sort_index()

    # 2. Bikin DataFrame rapi
    tabel = pd.DataFrame({
        'Rentang Nilai': counts.index,
        'Jumlah Customer': counts.values
    })

    # 3. Hitung Persentase biar Lebih kebayang
    tabel['Persentase (%)'] = (tabel['Jumlah Customer'] / tabel['Jumlah Customer']).

    print(f"--- TABEL DISTRIBUSI: {nama_kolom} ---")
    display(tabel)
    print("\n")

# Panggil fungsinya buat 3 kolom RFM
buat_tabel_distribusi('MonetaryValue', 'MONETARY (Total Belanja)')
buat_tabel_distribusi('Frequency', 'FREQUENCY (Seringnya Belanja)')
buat_tabel_distribusi('Recency', 'RECENCY (Hari Terakhir Belanja)')

--- TABEL DISTRIBUSI: MONETARY (Total Belanja) ---
```

	Rentang Nilai	Jumlah Customer	Persentase (%)
0	(-347.613, 34917.83]	4265	99.53
1	(34917.83, 69834.11]	12	0.28
2	(69834.11, 104750.39]	3	0.07
3	(104750.39, 139666.67]	1	0.02
4	(139666.67, 174582.95]	1	0.02
5	(174582.95, 209499.23]	1	0.02
6	(209499.23, 244415.51]	0	0.00
7	(244415.51, 279331.79]	1	0.02
8	(279331.79, 314248.07]	0	0.00
9	(314248.07, 349164.35]	1	0.02

--- TABEL DISTRIBUSI: FREQUENCY (Seringnya Belanja) ---

	Rentang Nilai	Jumlah Customer	Persentase (%)
0	(0.818, 19.2]	4187	97.71
1	(19.2, 37.4]	62	1.45
2	(37.4, 55.6]	19	0.44
3	(55.6, 73.8]	6	0.14
4	(73.8, 92.0]	4	0.09
5	(92.0, 110.2]	3	0.07
6	(110.2, 128.4]	1	0.02
7	(128.4, 146.6]	1	0.02
8	(146.6, 164.8]	1	0.02
9	(164.8, 183.0]	1	0.02

--- TABEL DISTRIBUSI: RECENCY (Hari Terakhir Belanja) ---

	Rentang Nilai	Jumlah Customer	Persentase (%)
0	(-0.373, 37.3]	1814	42.33
1	(37.3, 74.6]	876	20.44
2	(74.6, 111.9]	382	8.91
3	(111.9, 149.2]	234	5.46
4	(149.2, 186.5]	221	5.16
5	(186.5, 223.8]	184	4.29
6	(223.8, 261.1]	208	4.85
7	(261.1, 298.4]	145	3.38
8	(298.4, 335.7]	104	2.43
9	(335.7, 373.0]	117	2.73



Analisis Distribusi Data RFM

Berdasarkan visualisasi histogram dan tabel distribusi frekuensi di atas, berikut adalah temuan utama mengenai karakteristik data pelanggan:

1. Catatan Teknis: Nilai Negatif pada Rentang Awal

Pada tabel distribusi, terlihat batas bawah interval bernilai negatif (contoh: `-347.613` pada Monetary atau `-0.373` pada Recency).

- **Penjelasan:** Ini bukan data bernilai negatif. Ini adalah mekanisme teknis dari fungsi `pandas.cut` yang sedikit memperlebar batas bawah interval (*margin*) untuk memastikan nilai minimum (0) tetap terhitung masuk ke dalam keranjang (inklusif). Secara praktis, batas ini dapat dibaca mulai dari **0**.

2. Distribusi Monetary (Total Belanja)

- **Kondisi:** Sangat timpang (*Highly Skewed*).
- **Temuan:** Sebanyak **99.53%** pelanggan berada pada rentang belanja terendah (0 - 34.000). Hanya segelintir pelanggan ekstrem (*Outliers*) yang berbelanja hingga >300.000.
- **Implikasi:** Adanya outlier "Sultan" ini dapat mendistorsi pembentukan kluster pada algoritma K-Means jika tidak ditangani.

3. Distribusi Frequency (Frekuensi Belanja)

- **Kondisi:** Sangat timpang (*Highly Skewed*).

- **Temuan:** Mayoritas mutlak (**97.71%**) pelanggan berbelanja dalam frekuensi wajar (1 - 19 kali). Namun, terdapat outlier ekstrem (kemungkinan *reseller*) yang bertransaksi hingga >160 kali dalam satu periode.

4. Distribusi Recency (Kebaruan Transaksi)

- **Kondisi:** Terdistribusi lebih wajar (*Decent Distribution*).
- **Temuan:** Pola data menurun secara gradual. Sekitar **42.33%** pelanggan baru saja bertransaksi dalam 37 hari terakhir (pelanggan aktif). Pelanggan yang sudah lama tidak aktif (>300 hari) jumlahnya minoritas.
- **Implikasi:** Variabel ini memiliki sebaran yang cukup sehat untuk analisis klasterisasi tanpa perlu penanganan ekstrem.

Kesimpulan Awal: Variabel **Monetary** dan **Frequency** memerlukan pra-pemrosesan lebih lanjut (seperti transformasi logaritmik) untuk menormalkan sebaran data dan mengurangi dampak outlier sebelum dimasukkan ke dalam model K-Means.

Mungkin ide pertama kita, ingin langsung hapus aja ya kan outlier outlier itu. Nah akan tetapi dalam konteks ini, sebenarnya informasi itu akan berharga buat kita. Jika dilihat dan coba dianalisis mendasar, bisa dikatakan bahwa data itu mempresentasikan seorang customer yang paling banyak ngabisin uang dan bisa saja yang merupakan orang yang loyal? orang yang perlu dilakukan lebih istimewa (VIP) dan sebagainya.

Kita enggak ingin untuk menghapus customer-customer yang outlier ini karena mereka sebenarnya yang paling bervaluable buat kita. Kita fokuss pada tujuan utama kita apa, biarkan model yang menyesuaikan tujuan kita.

Starategi kita (Trent) adalah memisahkan dulu data outlier, kita analisis nanti dan sisanya akan kita gunakan untuk sebagai input bagi model KMeans kita.

```
In [94]: M_Q1 = aggregated_df["MonetaryValue"].quantile(0.25)
M_Q3 = aggregated_df["MonetaryValue"].quantile(0.75)
M_IQR = M_Q3 - M_Q1

monetary_outliers_df = aggregated_df[(aggregated_df["MonetaryValue"] > (M_Q3 + 1.5 *
monetary_outliers_df.describe()
```

Out[94]:

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	Recency
count	423.00	423.00	423.00	423	423.00
mean	15103.04	12188.10	17.17	2010-11-09 12:26:02.978723328	30.04
min	12357.00	3802.04	1.00	2009-12-10 18:03:00	0.00
25%	13622.00	4605.94	8.00	2010-11-08 13:17:30	3.00
50%	14961.00	6191.32	12.00	2010-11-26 12:19:00	13.00
75%	16692.00	10273.24	18.00	2010-12-06 10:34:30	31.00
max	18260.00	349164.35	183.00	2010-12-09 19:32:00	364.00
std	1728.66	25830.85	19.73	NaN	51.54

In [95]:

```

F_Q1 = aggregated_df["Frequency"].quantile(0.25)
F_Q3 = aggregated_df["Frequency"].quantile(0.75)
F_IQR = F_Q3 - F_Q1

frequency_outliers_df = aggregated_df[(aggregated_df["Frequency"] > (F_Q3 + 1.5 * F_IQR))
frequency_outliers_df.describe()

```

Out[95]:

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	Recency
count	279.00	279.00	279.00	279	279.00
mean	15352.66	14409.71	23.81	2010-11-23 11:06:20.645161216	16.09
min	12437.00	1094.39	12.00	2010-05-12 16:51:00	0.00
25%	13800.00	4331.56	13.00	2010-11-20 13:14:30	2.00
50%	15465.00	6615.77	17.00	2010-12-02 10:46:00	7.00
75%	16828.50	11692.41	23.00	2010-12-07 11:08:30	19.00
max	18260.00	349164.35	183.00	2010-12-09 19:32:00	211.00
std	1748.43	31381.74	21.93	NaN	26.59

Perlu dicatat, untuk kedua outlier ini sebenarnya bisa saja memiliki overlap satu sama lain. Secara intuitive, jika orang melakukan banyak pembelian (**Frequency** besar) maka bisa saja uang yang dihabiskan juga banyak (**Monetary** besar). Meskipun, ada kemungkinan juga, sering beli tapi dalam jumlah kecil

```
In [96]: # Mencari Customer ID yang ada di KEDUA tabel outlier
overlap_customers = set(monetary_outliers_df['Customer ID']).intersection(set(frequency_outliers_df['Customer ID']))

print(f"Jumlah Sultan (Monetary Outlier): {len(monetary_outliers_df)}")
print(f"Jumlah Si Rajin (Frequency Outlier): {len(frequency_outliers_df)}")
print(f"Jumlah 'Sultan Rajin' (Overlap): {len(overlap_customers)}")
```

Jumlah Sultan (Monetary Outlier): 423
 Jumlah Si Rajin (Frequency Outlier): 279
 Jumlah 'Sultan Rajin' (Overlap): 226

Good to Know :

Siap! Mari kita bedah rumus detektif pencari outlier ini: **IQR Method (Interquartile Range)**.

Ini adalah metode standar statistik untuk menentukan: "*Mana sih batas normal, dan mana yang udah kelewatan (ekstrem)?*"

Mari kita pakai analogi **Gaji Penduduk Desa (100 Orang)**.

Step 1: Urutkan Data (Sorting)

Bayangkan kita bariskan 100 orang dari yang gajinya paling kecil sampai paling besar.

- Orang ke-1: Gaji Rp 1.000
- ...
- Orang ke-50: Gaji Rp 5.000
- ...
- Orang ke-100: Gaji Rp 1.000.000 (Sultan)

Step 2: Cari Posisi Kuartil (Q1 dan Q3)

Kode: `aggregated_df["MonetaryValue"].quantile(0.25)`

1. **Q1 (Kuartil 1 / 25%)**: Kita cari orang di urutan ke-25.
 - Misal gajinya **Rp 3.000**.
 - Ini adalah batas bawah orang "biasa".
2. **Q3 (Kuartil 3 / 75%)**: Kita cari orang di urutan ke-75.
 - Misal gajinya **Rp 8.000**.
 - Ini adalah batas atas orang "biasa".

Step 3: Hitung Lebar Badan (IQR)

Kode: `M_IQR = M_Q3 - M_Q1`

IQR (Interquartile Range) adalah jarak antara si Q3 dan Q1.

- $IQR = 8.000 - 3.000 = 5.000$
- Artinya: "Sebagian besar orang (tengah-tengah) gajinya cuma selisih 5.000 perak."

Step 4: Tentukan Batas "Wajar" (The Fence)

Statistik punya aturan baku: "**Batas wajar adalah 1.5 kali lebar badan (IQR).**"

Kenapa 1.5? Itu aturan umum (Rule of Thumb) dari bapak statistik John Tukey. Angka ini dianggap pas buat nentuin mana yang "aneh".

1. Batas Atas Wajar:

$$Q3 + (1.5 \times IQR)$$

$$8.000 + (1.5 \times 5.000) = 8.000 + 7.500 = 15.500$$

- Artinya: Siapapun yang gajinya di atas Rp 15.500 dianggap **Outlier Kaya**.

2. Batas Bawah Wajar:

$$Q1 - (1.5 \times IQR)$$

$$3.000 - 7.500 = -4.500$$

- Artinya: Siapapun yang gajinya di bawah ini outlier miskin. (Tapi di kasus belanja, jarang ada minus, jadi batas bawah ini sering diabaikan kalau nilainya negatif).

Step 5: Filter Data (Eksekusi)

Kode: `aggregated_df[ ... > Batas Atas ...]`

Komputer akan mengecek satu-satu:

- Budi (Gaji 5.000) → Di bawah 15.500 → **Normal**.
- Siti (Gaji 10.000) → Di bawah 15.500 → **Normal**.
- **Sultan (Gaji 1.000.000)** → Di atas 15.500 → **OUTLIER! TANGKAP!**

Ringkasan Proses Kodingan Kamu:

1. `quantile(0.25)` : Cari angka batas bawah (Q1).
2. `quantile(0.75)` : Cari angka batas atas (Q3).
3. `IQR` : Hitung selisihnya.
4. `(M_Q3 + 1.5 * M_IQR)` : Hitung "Pagar Batas Atas".
5. `aggregated_df[ ... ]` : Ambil semua orang yang nilainya **lompat** melewati pagar tersebut.

Hasilnya (`monetary_outliers_df`) adalah daftar orang-orang yang "melompati pagar kewajaran" alias para Sultan. Jelas Mas?

Saya sangat amat perlu untuk mengulang dan belajar statistika lagi deh

```
In [97]: non_outliers_df = aggregated_df[~aggregated_df.index.isin(monetary_outliers_df.index)]
non_outliers_df.describe()
```

```
Out[97]:
```

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	Recency
count	3809.00	3809.00	3809.00	3809	3809.00
mean	15376.48	885.50	2.86	2010-09-03 11:16:46.516146176	97.08
min	12346.00	1.55	1.00	2009-12-01 10:49:00	0.00
25%	13912.00	279.91	1.00	2010-07-08 14:48:00	22.00
50%	15389.00	588.05	2.00	2010-10-12 16:25:00	58.00
75%	16854.00	1269.05	4.00	2010-11-17 13:14:00	154.00
max	18287.00	3788.21	11.00	2010-12-09 20:01:00	373.00
std	1693.20	817.67	2.24	NaN	98.11

The data is a lot better now

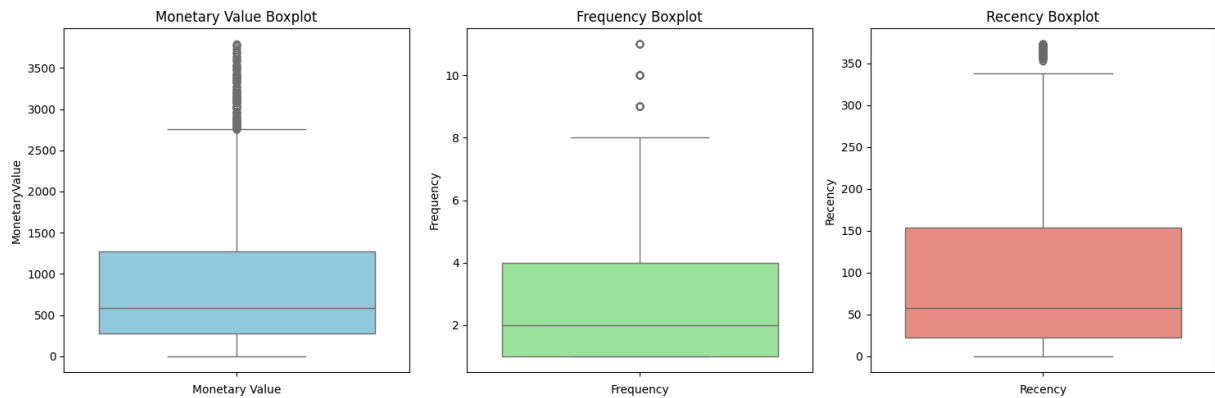
```
In [98]: # Metode paling cocok buat ngecek Outliers
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
sns.boxplot(data=non_outliers_df['MonetaryValue'], color='skyblue')
plt.title('Monetary Value Boxplot')
plt.xlabel('Monetary Value')

plt.subplot(1, 3, 2)
sns.boxplot(data=non_outliers_df['Frequency'], color='lightgreen')
plt.title('Frequency Boxplot')
plt.xlabel('Frequency')

plt.subplot(1, 3, 3)
sns.boxplot(data=non_outliers_df['Recency'], color='salmon')
plt.title('Recency Boxplot')
plt.xlabel('Recency')

plt.tight_layout()
plt.show()
```

Ini jauh lebih tolerable dibandingkan dengan yang sebelumnya

Analisis Grafik After Cleaning (Non-Outliers)

1. Monetary (Kiri - Biru):

- **Dulu:** Kotaknya gepeng di bawah, titiknya terbang sampai 350.000.
- **Sekarang:** Kotaknya (Box) kelihatan jelas dan besar! Batas atasnya (Kumis) ada di sekitar 2.700-an.
- **Outlier:** Masih ada titik-titik di atas kumis (sampai 3.700), tapi ini **WAJAR**. Jaraknya dekat. K-Means masih bisa menoleransi perbedaan 1.000 poin, tapi gak bisa menoleransi perbedaan 300.000 poin.

2. Frequency (Tengah - Hijau):

- **Dulu:** Titik sampai 175.
- **Sekarang:** Titik maksimal cuma 11.
- **Box:** Kelihatan jelas bahwa mayoritas orang belanja 1-4 kali. Ini data yang sangat *representatif* untuk populasi umum.

3. Recency (Kanan - Merah):

- **Kondisi:** Tidak banyak berubah karena dari awal Recency sudah cukup sehat. Tapi sekarang skalanya jadi lebih padat.

Makna "Tolerable Outliers"

Kenapa Trent bilang "*Masih ada outlier, tapi tolerable*"?

Di dunia nyata, data itu gak pernah mulus 100% kayak pantat bayi. Pasti ada "noise".

- Kalau kita paksakan buang SEMUA titik hitam itu sampai bersih, nanti data kita habis.
- Outlier yang sekarang (misal Monetary 3.500) itu masih dianggap "Customer Biasa yang agak boros dikit", BUKAN "Sultan".
- Mesin K-Means masih sanggup menghitung jarak segitu tanpa jadi bias parah.

In [99]: `non_outliers_df.head(10)`

Out[99]:

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	Recency
0	12346.00	169.36	2	2010-06-28 13:53:00	164
1	12347.00	1323.32	2	2010-12-07 14:57:00	2
2	12348.00	221.16	1	2010-09-27 14:59:00	73
3	12349.00	2221.14	2	2010-10-28 08:23:00	42
4	12351.00	300.93	1	2010-11-29 15:23:00	10
5	12352.00	343.80	2	2010-11-29 10:07:00	10
6	12353.00	317.76	1	2010-10-27 12:44:00	43
7	12355.00	488.21	1	2010-05-21 11:59:00	202
8	12356.00	3126.25	3	2010-11-24 12:24:00	15
10	12358.00	2519.01	3	2010-11-29 10:56:00	10

In [100...]

```
fig = plt.figure(figsize=(10, 10))
ax = fig.add_subplot(projection='3d') # 111 artinya: 1 baris, 1 kolom, urutan 1

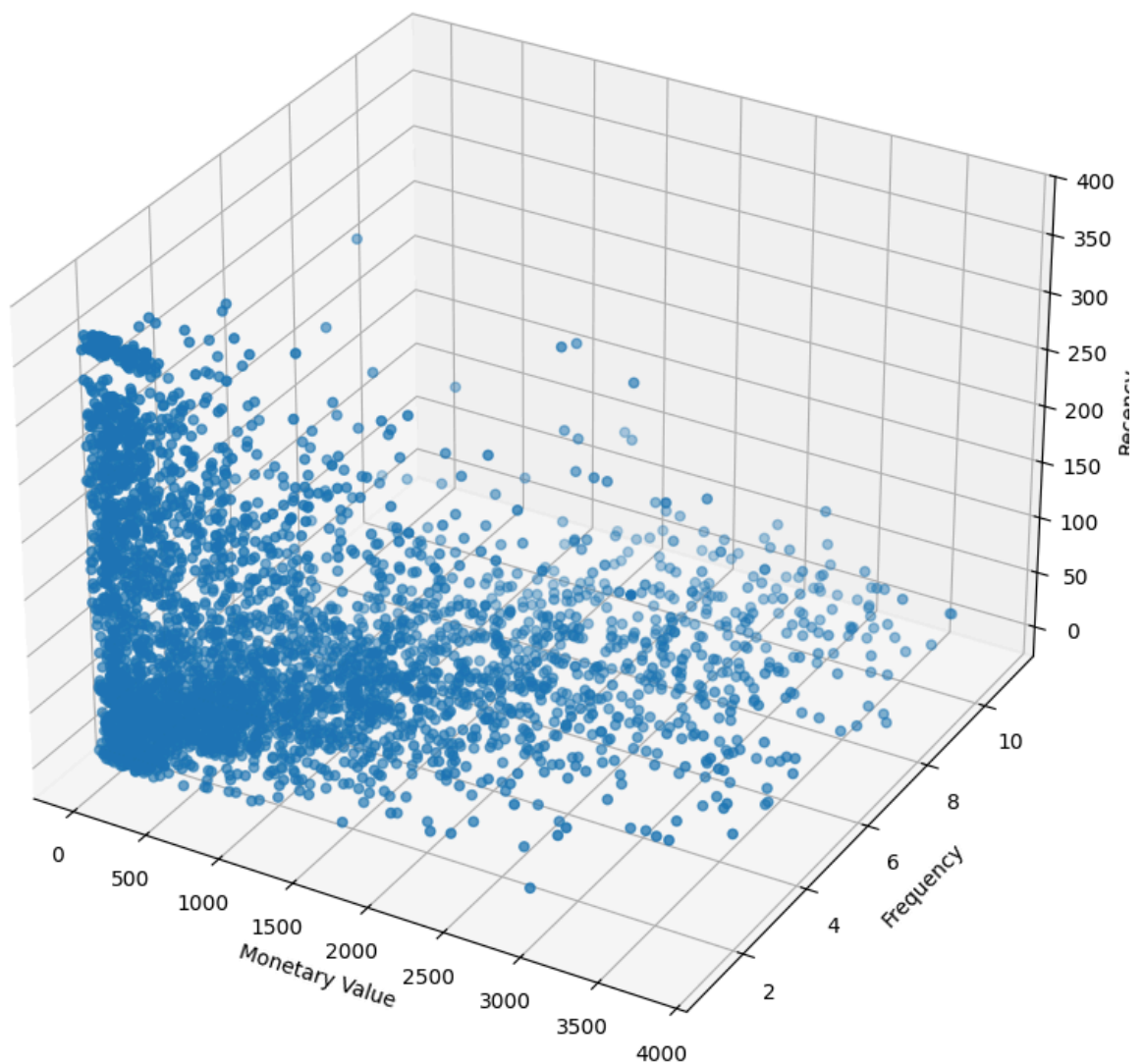
scatter = ax.scatter(non_outliers_df["MonetaryValue"], non_outliers_df["Frequency"])

ax.set_xlabel('Monetary Value')
ax.set_ylabel('Frequency')
ax.set_zlabel('Recency')

ax.set_title('3D Scatter Plot of Customer Data')

plt.show()
```

3D Scatter Plot of Customer Data



Dapat dilihat, bahwasannya setiap axes memiliki perbedaan skala, misalnya monetary value yang dimana nilainya rentang 500, sedangkan frequency hanya rentang 2 dan sebagainya

⚠ Visualisasi Masalah Skala (Scaling Issue)

Berdasarkan *3D Scatter Plot* di atas, terlihat jelas adanya ketimpangan skala antar variabel (*Feature Scaling Problem*):

- **Monetary (Sumbu X):** Memiliki rentang nilai dalam ribuan (0 - 3.500+).
- **Recency (Sumbu Z):** Memiliki rentang nilai dalam ratusan (0 - 370).
- **Frequency (Sumbu Y):** Memiliki rentang nilai satuan (1 - 11).

Dampak pada K-Means: Algoritma K-Means menggunakan perhitungan **Jarak Euclidean** untuk menentukan kedekatan antar titik data. Tanpa normalisasi skala, algoritma akan menjadi **bias** terhadap variabel dengan nilai nominal terbesar (Monetary). Perbedaan kecil pada Monetary (misal: 500 poin) akan dianggap jauh lebih signifikan daripada perbedaan pada Frequency (misal: 5 poin), padahal secara bisnis, frekuensi belanja sangat krusial.

Kenapa ini Masalah Besar buat K-Means?

K-Means itu ngitung jarak pakai rumus Pythagoras (Jarak Euclidean).

$$Jarak = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

Bayangkan:

- Beda 1 poin di Frequency $\rightarrow 1^2 = 1$. (Kecil banget).
- Beda 1000 poin di Monetary $\rightarrow 1000^2 = 1.000.000$. (Gede banget).

Akibatnya: K-Means bakal **MENGABAIKAN Frequency**. Dia bakal mikir: "Ah, Frequency cuma beda 1-2 angka doang, gak penting. Monetary bedanya ribuan, ini yang penting!"

Padahal secara bisnis, beda belanja 1 kali vs 10 kali itu signifikan banget (Loyal vs Coba-coba).

Solusi: Kita wajib melakukan **Standardisasi Data (Scaling)** menggunakan

`StandardScaler` untuk mengubah distribusi data menjadi skala standar (Mean = 0, Std Dev = 1). Hal ini memastikan setiap variabel memiliki bobot yang setara dalam proses klasterisasi.

Untuk mengatasi masalah perbedaan skala antar variabel (Monetary dalam ribuan vs Frequency dalam satuan), kita menerapkan teknik **Standard Scaling** (disebut juga Z-score Normalization).

Metode ini mentransformasi nilai asli data sehingga distribusi data yang baru akan memiliki **Rata-rata (Mean) = 0** dan **Standar Deviasi = 1**. Hal ini menjamin bahwa setiap fitur memberikan kontribusi yang setara (*equal weight*) dalam perhitungan jarak K-Means.

Rumus Matematis

$$z = \frac{x - \mu}{\sigma}$$

Keterangan:

- z : Nilai hasil standardisasi (*Standardized value / Z-score*).
- x : Nilai asli data (*Original value*).
- μ (μ) : Rata-rata (*Mean*) dari fitur tersebut.
- σ (σ) : Standar Deviasi dari fitur tersebut.

💡 Ilustrasi Studi Kasus (Mengapa ini Penting?)

Mari kita simulasikan bagaimana rumus ini bekerja pada data RFM kita agar menjadi setara.

Kasus: Kita memiliki Customer A yang sangat loyal.

1. **Monetary (Uang):** Dia belanja **3.000**. (Rata-rata populasi = 1.500, Standar Deviasi = 500).
2. **Frequency (Frekuensi):** Dia belanja **10 kali**. (Rata-rata populasi = 4 kali, Standar Deviasi = 2).

Tanpa Scaling: K-Means akan melihat angka **3.000** jauh lebih besar daripada **10**. Fitur Frequency akan dianggap tidak penting.

Dengan Standard Scaling (Hitungan Manual):

1. **Hitung Z-Score Monetary:**

$$z = \frac{3000 - 1500}{500} = \frac{1500}{500} = \mathbf{3.0}$$

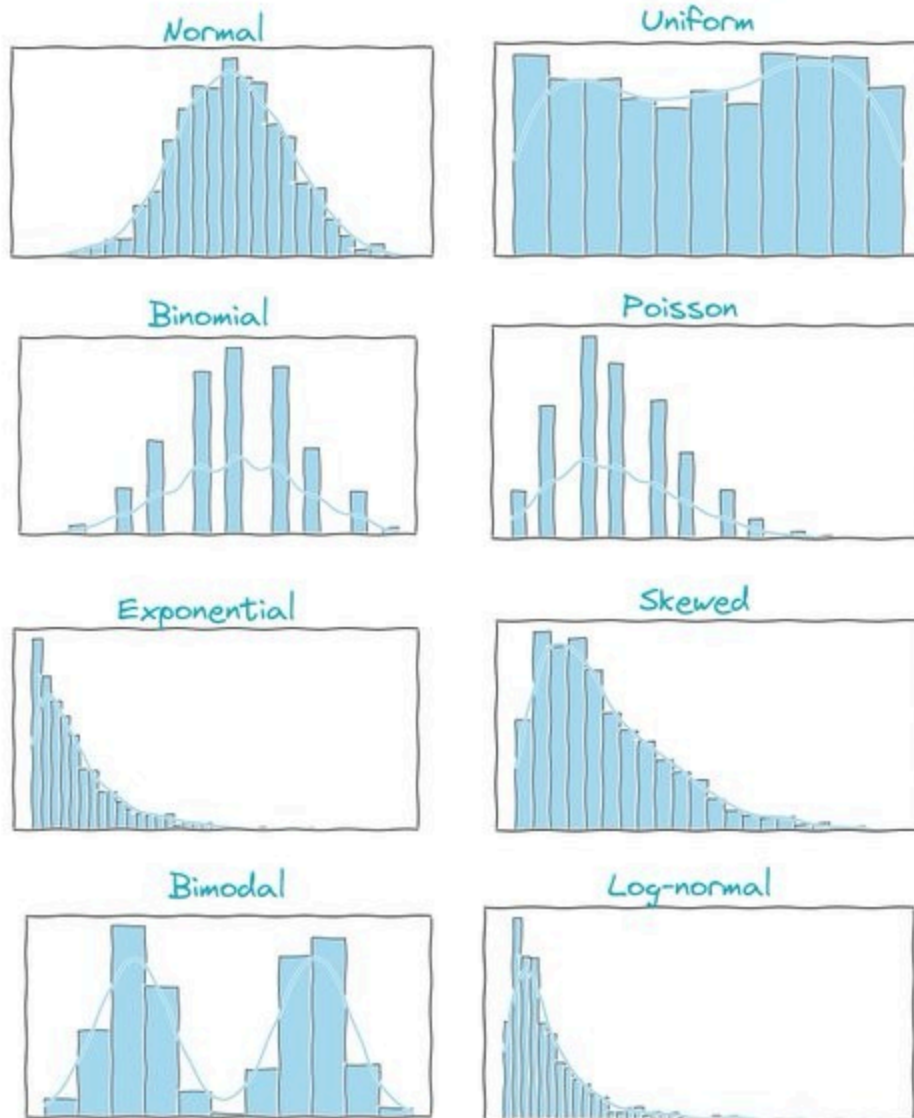
2. **Hitung Z-Score Frequency:**

$$z = \frac{10 - 4}{2} = \frac{6}{2} = \mathbf{3.0}$$

Kesimpulan: Setelah di-scaling, komputer melihat bahwa kontribusi Monetary (Nilai 3.0) dan Frequency (Nilai 3.0) adalah **SETARA**. Keduanya sama-sama "3 poin di atas rata-rata". Inilah kondisi ideal untuk algoritma K-Means.

Ada beberapa hal yang perlu dicatat, bahwasannyaa dalam hal ini, kitaa mengansumsi saja untuk distribusi data kita memiliki distribusi normal.

Data distributions



 @daansan_ml

```

In [101... scaler = StandardScaler()

scaled_data = scaler.fit_transform(non_outliers_df[["MonetaryValue", "Frequency", "
scaled_data

Out[101... array([[ -0.87594534, -0.38488934,  0.68214853],
        [ 0.5355144 , -0.38488934, -0.96925093],
        [-0.81258645, -0.83063076, -0.24548944],
        ...,
        [-0.62197163, -0.83063076,  2.01753946],
        [ 0.44146683, -0.38488934,  0.14187587],
        [ 1.72488781,  0.50659348, -0.81634357]], shape=(3809, 3))

```

Proses nya gimana sih??

Tentu! Mari kita bongkar "Sihir Matematika" di balik layar `StandardScaler` dan `fit_transform`.

Kita pakai contoh data sederhana (3 Pelanggan) biar gampang dibayangkan.

Data Awal (`non_outliers_df`):

Customer	Monetary (M)	Frequency (F)
A	1000	2

Customer	Monetary (M)	Frequency (F)
B	2000	4
C	3000	6

Tahap 1: `scaler.fit` (Belajar dari Data)

Saat kamu panggil `fit`, komputer akan "mempelajari" karakteristik kolom M dan F secara terpisah. Dia menghitung **Rata-rata** (μ) dan **Standar Deviasi** (σ).

Perhitungan Otak Komputer:

1. Untuk Kolom Monetary (M):

- Rata-rata (μ_M) = $(1000 + 2000 + 3000)/3 = 2000$
- Anggap saja Standar Deviasi (σ_M) = 1000 (biar gampang hitungnya).

2. Untuk Kolom Frequency (F):

- Rata-rata (μ_F) = $(2 + 4 + 6)/3 = 4$
- Anggap saja Standar Deviasi (σ_F) = 2.

Sekarang Scaler sudah punya "Kunci Jawaban" (μ dan σ) di memori.

Tahap 2: `.transform` (Mengubah Data)

Komputer menerapkan rumus sakti $z = \frac{x-\mu}{\sigma}$ ke **Setiap Sel Data**.

Transformasi Customer A:

- **Monetary Baru:** $\frac{1000-2000}{1000} = \frac{-1000}{1000} = -1.0$
- **Frequency Baru:** $\frac{2-4}{2} = \frac{-2}{2} = -1.0$

Transformasi Customer B:

- **Monetary Baru:** $\frac{2000-2000}{1000} = 0.0$
- **Frequency Baru:** $\frac{4-4}{2} = 0.0$

Transformasi Customer C:

- **Monetary Baru:** $\frac{3000-2000}{1000} = 1.0$
- **Frequency Baru:** $\frac{6-4}{2} = 1.0$

Tahap 3: Hasil Akhir (`scaled_data_df`)

Tabel kamu berubah wujud menjadi seperti ini:

Customer	Monetary (Z)	Frequency (Z)
A	-1.0	-1.0
B	0.0	0.0
C	1.0	1.0

Apa Artinya?

- **Angka Negatif (-1.0):** Artinya nilai customer ini **DI BAWAH RATA-RATA**.
- **Angka Nol (0.0):** Artinya nilai customer ini **PERSIS RATA-RATA**.
- **Angka Positif (1.0):** Artinya nilai customer ini **DI ATAS RATA-RATA**.

Lihat `scaled_data_df` kamu:

- Baris 0 (Customer 12346): Monetary **-0.88**.
 - *Artinya:* Dia belanja di bawah rata-rata (customer kecil).
- Baris 3 (Customer 12349): Monetary **1.63**.
 - *Artinya:* Dia belanja jauh di atas rata-rata (customer lumayan besar).

Sekarang, skala angka mereka sudah setara (berkisar antara -2 sampai +2), jadi K-Means bisa menilai mereka dengan adil tanpa tertipu nominal ribuan.

In [102...

```
scaled_data_df = pd.DataFrame(scaled_data, index=non_outliers_df.index, columns=["M", "F", "R"])
scaled_data_df
```

Out[102...

	MonetaryValue	Frequency	Recency
0	-0.88	-0.38	0.68
1	0.54	-0.38	-0.97
2	-0.81	-0.83	-0.25
3	1.63	-0.38	-0.56
4	-0.72	-0.83	-0.89
...	...	...	...
4280	-0.30	1.40	-0.82
4281	-0.58	-0.83	-0.32
4282	-0.62	-0.83	2.02
4283	0.44	-0.38	0.14
4284	1.72	0.51	-0.82

3809 rows × 3 columns

```

In [103... fig = plt.figure(figsize=(10, 10))
ax = fig.add_subplot(projection='3d') # 111 artinya: 1 baris, 1 kolom, urutan 1

scatter = ax.scatter(scaled_data_df["MonetaryValue"], scaled_data_df["Frequency"],

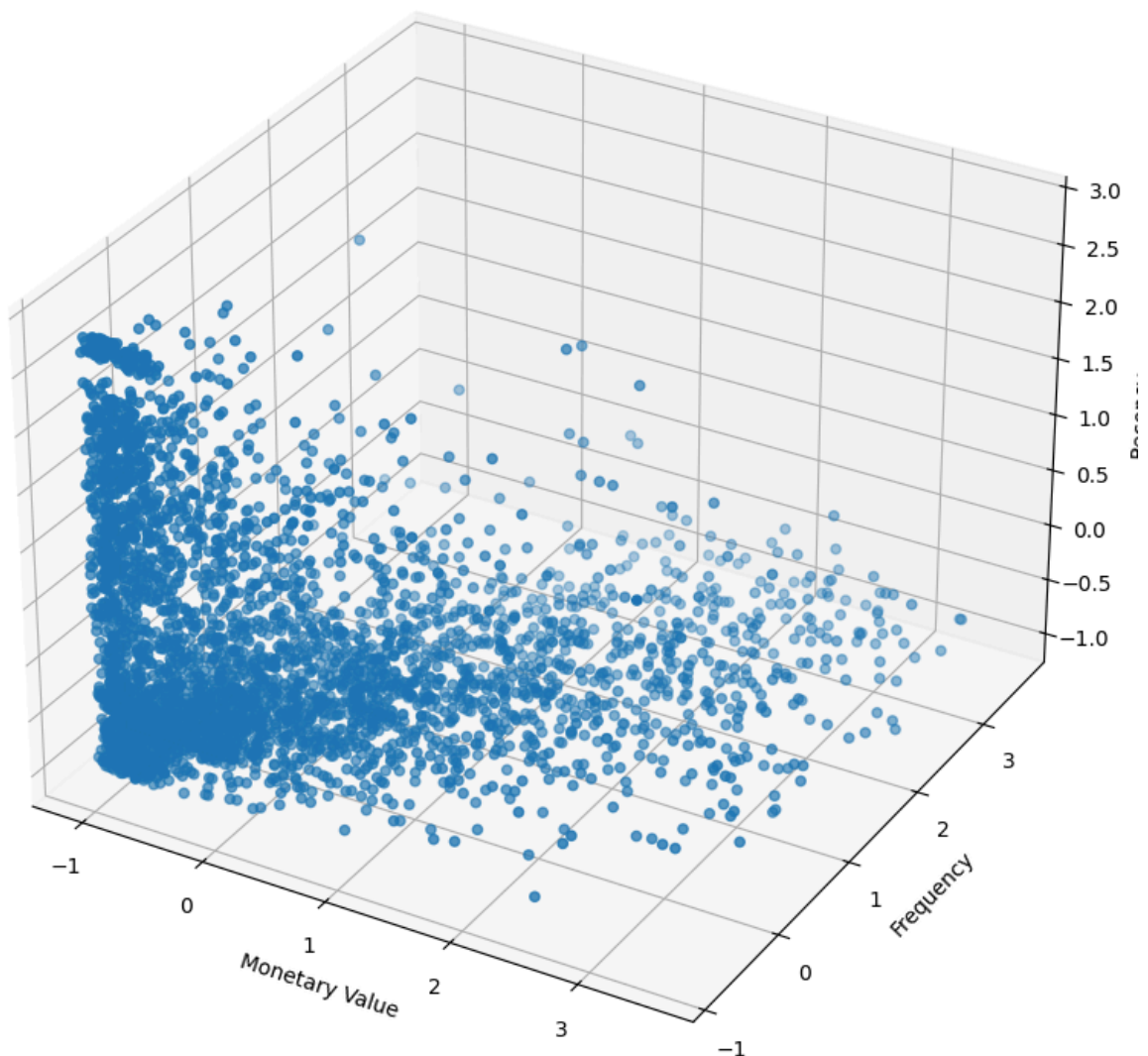
ax.set_xlabel('Monetary Value')
ax.set_ylabel('Frequency')
ax.set_zlabel('Recency')

ax.set_title('3D Scatter Plot of Customer Data')

plt.show()

```

3D Scatter Plot of Customer Data



Sekarang setiap data memiliki bobot yang sama

KMeans Clustering

```
In [104... max_k = 12

inertia = []
k_values = range(2, max_k + 1)

for k in k_values:

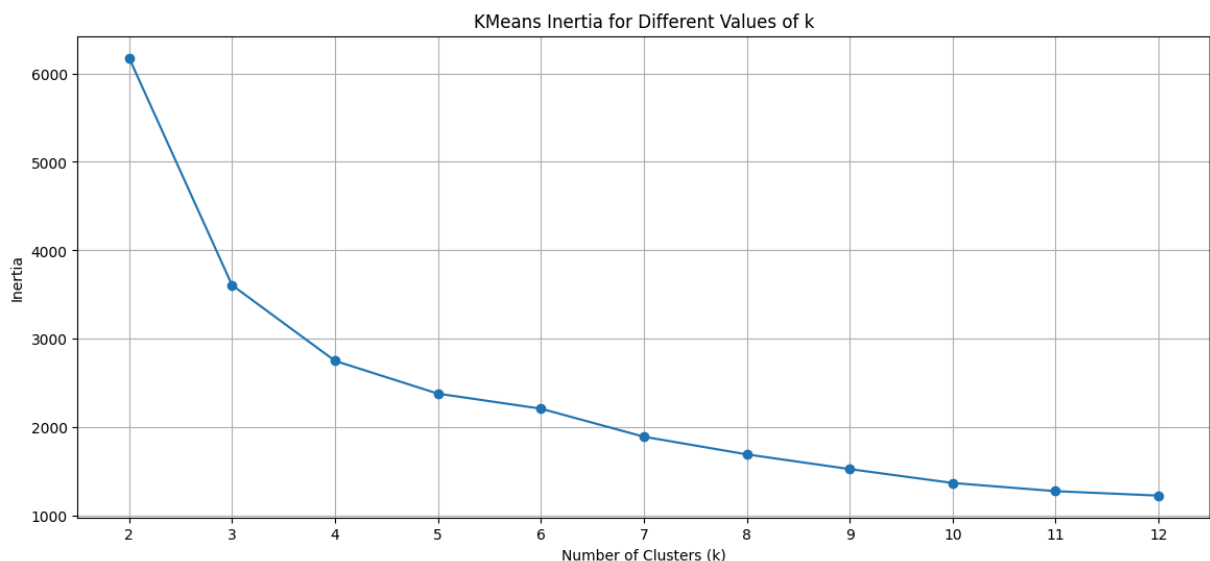
    kmeans = KMeans(n_clusters=k, random_state=42, max_iter=1000)

    kmeans.fit_predict(scaled_data_df)

    inertia.append(kmeans.inertia_)

plt.figure(figsize=(14, 6))

plt.plot(k_values, inertia, marker='o')
plt.title('KMeans Inertia for Different Values of k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.xticks(k_values)
plt.grid(True)
```



Dengan menggunakan metode elbow, menurut saya ini berada di antara 4 dan 5, karena setelahnya itu, tidak ada perubahan yang besar dan performance yang berdampak. Di antara K = 4 atau 5 adalah poin stability nya

Kodingan ini adalah ritual wajib untuk menjawab pertanyaan: "*Berapa sih jumlah kelompok yang pas?*"

Step 1: Menyiapkan Wadah dan Rentang K

```
max_k = 12
inertia = []
k_values = range(2, max_k + 1)
```

- `max_k = 12` : Kita mau mencoba membagi customer maksimal jadi 12 kelompok. (Bisa 15, bisa 20, tapi biasanya di atas 10 udah kebanyakan buat marketing).
- `inertia = []` : Ini adalah **Keranjang Kosong**. Nanti setiap kali kita hitung skor (Inertia) untuk K tertentu, skornya kita lempar ke sini.
- `range(2, max_k + 1)` : Kita akan mencoba dari **K=2** sampai **K=12**.
 - *Kenapa mulai dari 2?* Karena kalau K=1 (semua orang jadi satu kelompok) ya gak usah dicluster dong.

Step 2: Looping (Percobaan Berulang)

```
for k in k_values:
```

Ini adalah perintah **Perulangan**. Komputer akan melakukan hal yang sama berkali-kali untuk `k = 2`, lalu `k = 3`, sampai `k = 12`.

Apa yang dilakukan di dalam Loop?

A. Membuat Robot K-Means

```
kmeans = KMeans(n_clusters=k, random_state=42, max_iter=1000)
```

- `n_clusters=k` : "Eh Robot, sekarang coba bagi datanya jadi **2** kelompok ya." (Nanti putaran berikutnya jadi 3, dst).
- `random_state=42` : Ini "Kunci Jawaban". Supaya kalau kamu run ulang besok, hasilnya tetap sama (konsisten). Angka 42 itu sembarang, bisa berapa aja.
- `max_iter=1000` : "Robot, kamu boleh geser-geser centroid maksimal 1000 kali. Kalau belum ketemu juga, nyerah aja." (Biar gak looping selamanya).

B. Melatih Robot

```
kmeans.fit_predict(scaled_data_df)
```

- Di sini si Robot mulai bekerja: Lempar centroid → Hitung jarak → Geser centroid.
- Dia mempelajari pola data `scaled_data_df` kamu.

C. Menyimpan Skor (Raport)

```
inertia.append(kmeans.inertia_)
```

- `kmeans.inertia_` : Setelah robot selesai kerja, kita tanya: "*Gimana Bot? Seberapa rapi barisannya?*"
 - Ingat konsep **Variance/Inertia** tadi? Semakin kecil angkanya, semakin rapi/padat klasternya.
- `.append(...)` : Skor itu dimasukkan ke keranjang `inertia` yang tadi kita siapkan.

Step 3: Visualisasi (Menggambar Grafik)

Bagian `plt.plot(...)` dan seterusnya itu standar menggambar grafik garis.

- Sumbu X (`k_values`): Jumlah Kelompok (2, 3, 4...).
- Sumbu Y (`inertia`): Skor Kerapian (Makin ke bawah makin bagus).

Evaluasi Klaster: Silhouette Score

In [105...

```
max_k = 12

inertia = []
silhouette_scores = []
k_values = range(2, max_k + 1)

for k in k_values:

    kmeans = KMeans(n_clusters=k, random_state=42, max_iter=1000)

    clueter_labels = kmeans.fit_predict(scaled_data_df)

    sil_score = silhouette_score(scaled_data_df, clueter_labels)

    silhouette_scores.append(sil_score)

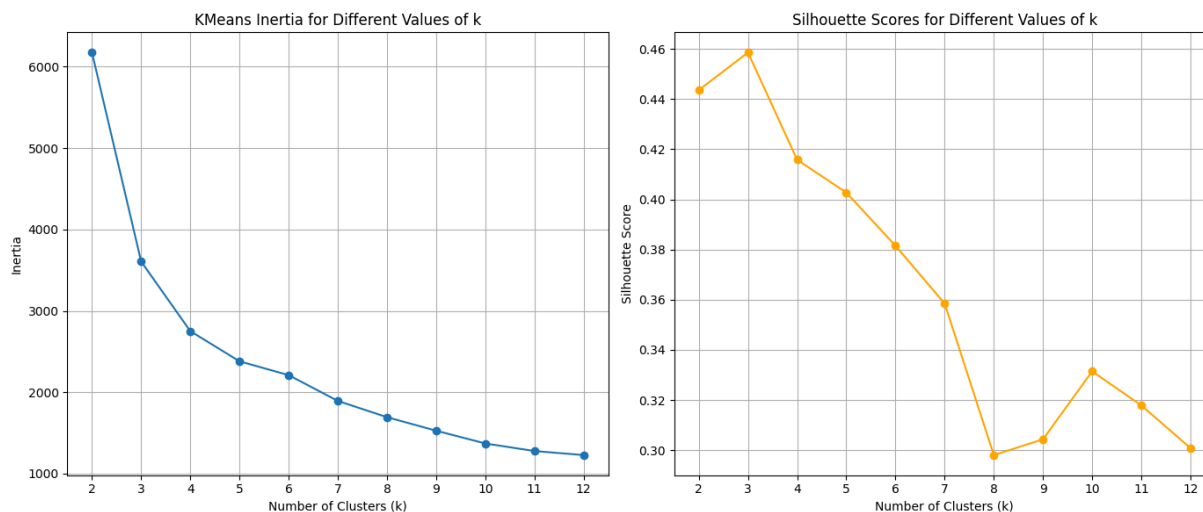
    inertia.append(kmeans.inertia_)

plt.figure(figsize=(14, 6))

plt.subplot(1, 2, 1)
plt.plot(k_values, inertia, marker='o')
plt.title('KMeans Inertia for Different Values of k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.xticks(k_values)
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(k_values, silhouette_scores, marker='o', color='orange')
plt.title('Silhouette Scores for Different Values of k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Silhouette Score')
plt.xticks(k_values)
plt.grid(True)

plt.tight_layout()
plt.show()
```



Selain menggunakan metode Elbow (Inertia), kita juga menggunakan metrik **Silhouette Score** untuk memvalidasi kualitas kluster yang terbentuk. Metode ini memberikan ukuran seberapa baik setiap objek diklasifikasikan dengan melihat dua aspek sekaligus: **Kepadatan (Cohesion)** dan **Pemisahan (Separation)**.

Rumus Matematis

Skor Silhouette untuk satu titik data i dihitung dengan rumus:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Keterangan:

- $s(i)$: Skor Silhouette (Rentang nilai: -1 sampai 1).
- $a(i)$: Rata-rata jarak antara titik i dengan semua titik lain **di dalam kluster yang sama** (Intra-cluster distance). *Semakin kecil $a(i)$, semakin kompak.*
- $b(i)$: Rata-rata jarak antara titik i dengan semua titik di **kluster tetangga terdekat** (Nearest-cluster distance). *Semakin besar $b(i)$, semakin terpisah jauh dari tetangga.*

Interpretasi Nilai:

- **Mendekati +1**: Sangat Bagus. Titik data berada jauh dari kluster tetangga dan sangat dekat dengan kluster sendiri.
- **0**: Titik data berada di perbatasan antar dua kluster (Ambigu).
- **Negatif (-1)**: Kemungkinan Salah Kamar (Misclassified). Titik data lebih dekat ke tetangga daripada ke kelompoknya sendiri.



Ilustrasi Studi Kasus (Logika Perhitungan)

Mari kita bayangkan situasi di sebuah Sekolah.

- **Siswa A** masuk di **Kelas IPA**.
- **Kelas Tetangga** adalah **Kelas IPS**.

1. Menghitung $a(i)$ - Jarak ke Teman Sekelas:

- Siswa A mengukur jarak tempat duduknya ke semua teman-teman di Kelas IPA.
- Ternyata rata-rata jaraknya **2 meter**. (Ini $a(i)$).
- *Artinya*: Dia duduk cukup kumpul dengan teman sekelasnya.

2. Menghitung $b(i)$ - Jarak ke Kelas Sebelah:

- Siswa A mengukur jarak tempat duduknya ke semua murid di Kelas IPS (Tetangga terdekat).
- Ternyata rata-rata jaraknya **10 meter**. (Ini $b(i)$).
- *Artinya*: Dia duduk jauh dari anak-anak IPS.

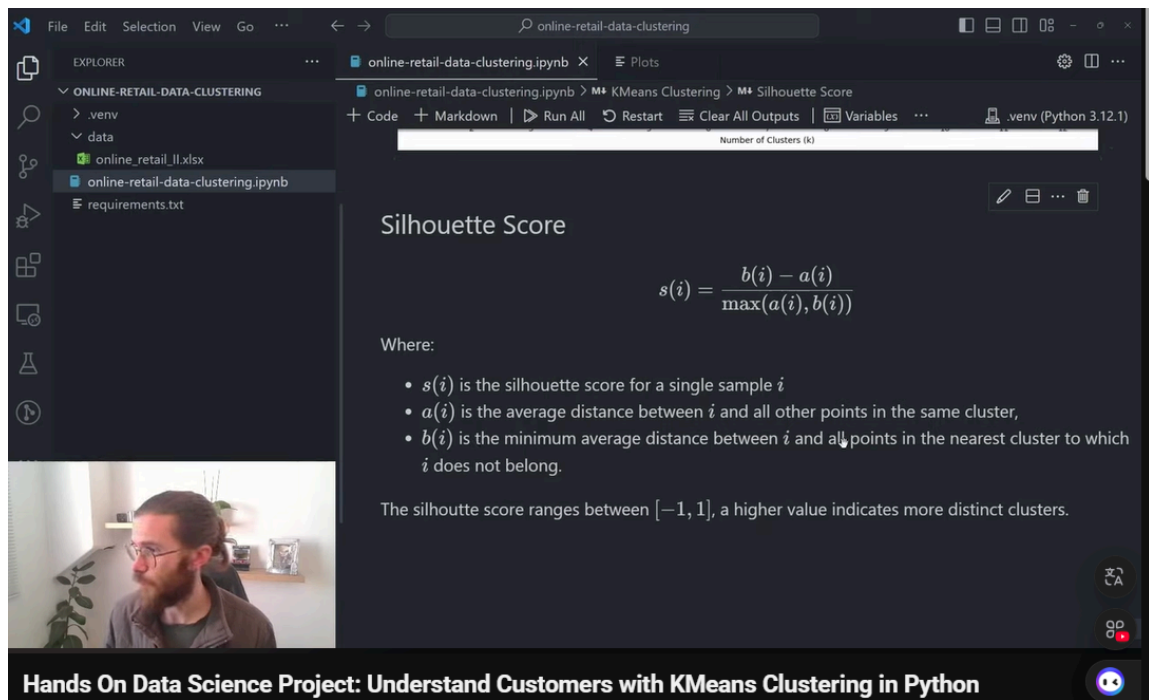
3. Menghitung Skor Silhouette:

$$s(i) = \frac{10 - 2}{\max(2, 10)} = \frac{8}{10} = 0.8$$

Kesimpulan: Skor **0.8** (Mendekati 1) berarti Siswa A sudah berada di posisi yang **SANGAT TEPAT**. Dia sangat dekat dengan teman sekelasnya (IPA) dan sangat jauh dari anak kelas sebelah (IPS). Klasterisasinya valid.

- **Mendekati +1 (Bagus Banget):** Clusternya kompak dan terpisah jauh dari cluster lain. (Bayangkan pulau-pulau yang terpisah lautan luas).
- **Sekitar 0 (Biasa Aja):** Clusternya mepet-mepet atau tumpang tindih (*overlap*). (Bayangkan perbatasan desa yang rumahnya nyambung).
- **Mendekati -1 (Jelek/Salah):** Datanya nyasar. Dia masuk Cluster A padahal lebih dekat ke Cluster B.

Jadi nanti kalau kita jalankan kode Silhouette Score, kita cari nilai K yang skor rata-ratanya **paling tinggi (paling dekat ke 1)**.



Silhouette Score

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Where:

- $s(i)$ is the silhouette score for a single sample i
- $a(i)$ is the average distance between i and all other points in the same cluster,
- $b(i)$ is the minimum average distance between i and all points in the nearest cluster to which i does not belong.

The silhouette score ranges between $[-1, 1]$, a higher value indicates more distinct clusters.

Hands On Data Science Project: Understand Customers with KMeans Clustering in Python

```
In [106... kmeans = KMeans(n_clusters=4, random_state=42, max_iter=1000)

cluster_labels = kmeans.fit_predict(scaled_data_df)

cluster_labels
```

```
Out[106... array([1, 0, 2, ..., 1, 0, 0], shape=(3809,), dtype=int32)
```

```
In [107... non_outliers_df["Cluster"] = cluster_labels

non_outliers_df
```

C:\Users\MSI KATANA 15\AppData\Local\Temp\ipykernel_18036\3577770544.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
`non_outliers_df["Cluster"] = cluster_labels`

Out[107...

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	Recency	Cluster
0	12346.00	169.36	2	2010-06-28 13:53:00	164	1
1	12347.00	1323.32	2	2010-12-07 14:57:00	2	0
2	12348.00	221.16	1	2010-09-27 14:59:00	73	2
3	12349.00	2221.14	2	2010-10-28 08:23:00	42	0
4	12351.00	300.93	1	2010-11-29 15:23:00	10	2
...	...	...	...	...	...	...
4280	18283.00	641.77	6	2010-11-22 15:30:00	17	0
4281	18284.00	411.68	1	2010-10-04 11:33:00	66	2
4282	18285.00	377.00	1	2010-02-17 10:24:00	295	1
4283	18286.00	1246.43	2	2010-08-20 11:57:00	111	0
4284	18287.00	2295.71	4	2010-11-22 11:51:00	17	0

3809 rows × 6 columns

In [108...

```

cluster_colors = {0: '#1f77b4', # Blue
                  1: '#ff7f0e', # Orange
                  2: '#2ca02c', # Green
                  3: '#d62728'} # Red

colors = non_outliers_df['Cluster'].map(cluster_colors)

fig = plt.figure(figsize=(10, 10))
ax = fig.add_subplot(projection='3d')

scatter = ax.scatter(non_outliers_df['MonetaryValue'],
                    non_outliers_df['Frequency'],
                    non_outliers_df['Recency'],
                    c=colors, # Use mapped solid colors
                    marker='o')

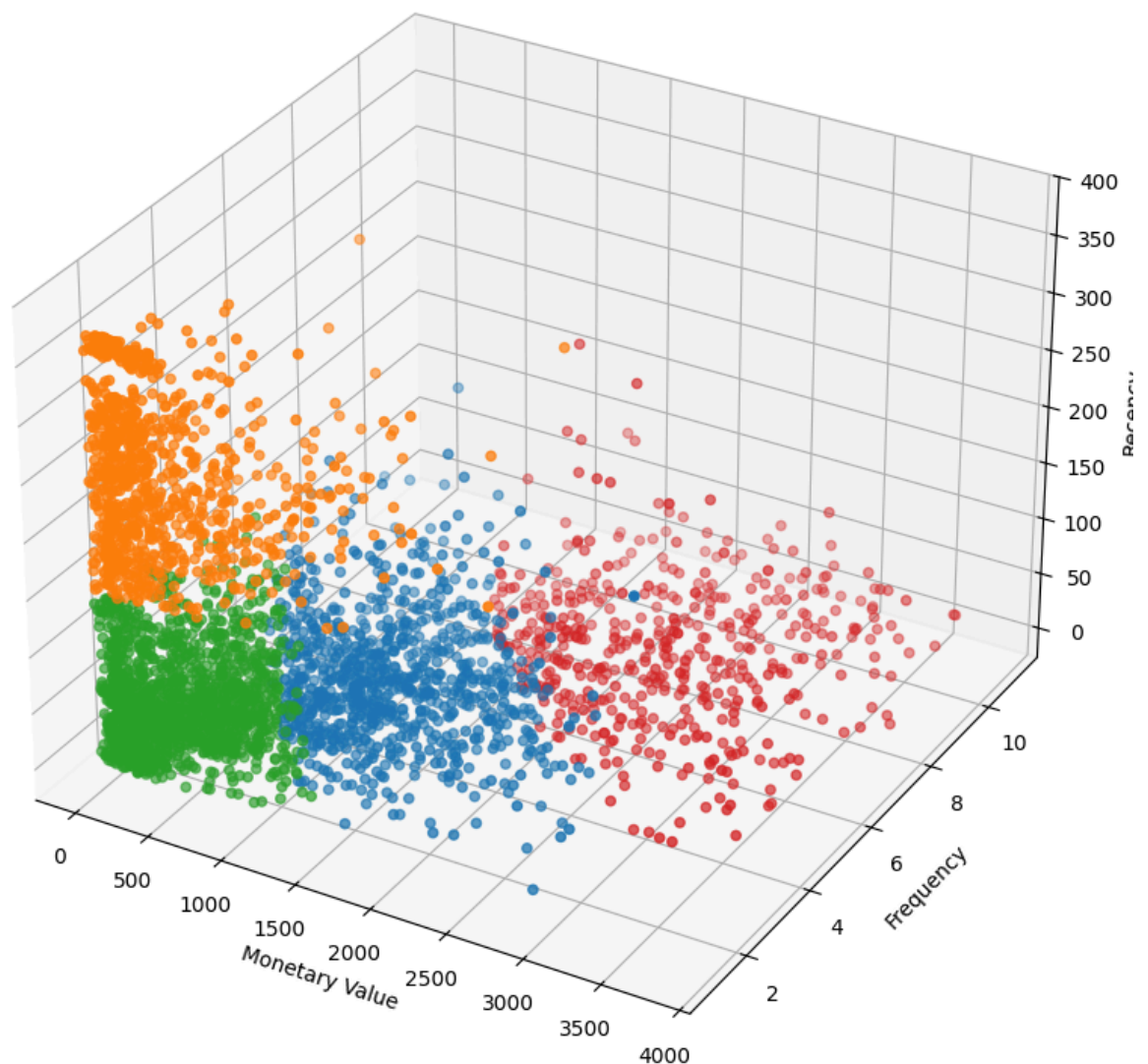
ax.set_xlabel('Monetary Value')
ax.set_ylabel('Frequency')
ax.set_zlabel('Recency')

ax.set_title('3D Scatter Plot of Customer Data by Cluster')

plt.show()

```

3D Scatter Plot of Customer Data by Cluster



Mari kita bongkar proses pembentukan klaster dan visualisasinya langkah demi langkah. Ini adalah tahap di mana data kamu akhirnya "dikasih label".

Langkah 1: Membentuk Pasukan Robot K-Means

```
kmeans = KMeans(n_clusters=4, random_state=42, max_iter=1000)
```

- `KMeans(...)` : Memanggil arsitek K-Means.
- `n_clusters=4` : Kamu memberi perintah tegas: "Saya sudah memutuskan, bagi data ini jadi **4 Kelompok** saja!" (Hasil keputusan dari Elbow & Silhouette tadi).
- `random_state=42` : Kunci biar hasilnya konsisten (kalau dijalankan ulang, orang yang masuk Kelompok A tetap di A).
- `max_iter=1000` : Memberi batas waktu berpikir biar komputer gak *hang* kalau datanya rumit.

Langkah 2: Melatih & Memprediksi (Fit & Predict)

```
cluster_labels = kmeans.fit_predict(scaled_data_df)
```

- **scaled_data_df** : Ingat, kita pakai data yang **SUDAH DI-SCALING** (yang angkanya -1 sampai 3 tadi), bukan data asli yang ribuan.
- **fit** : Robot mempelajari pola sebaran datanya.
- **predict** : Robot menunjuk satu-satu:
 - "Kamu Customer 1, masuk Kelompok 0!"
 - "Kamu Customer 2, masuk Kelompok 2!"
- **Output cluster_labels** : Ini adalah list angka `[1, 0, 2, 1, 0...]` yang isinya nomor kelompok masing-masing customer.

Langkah 3: Menempelkan Label ke Data Asli

```
non_outliers_df["Cluster"] = cluster_labels
```

- Ini langkah penting! Kita ambil label hasil prediksi tadi, lalu kita tempelkan balik ke tabel **Data Asli (Non-Scaled)**.
- **Kenapa ke data asli?** Karena kita mau baca profilnya pakai angka manusia (Rupiah/Dolar), bukan angka Z-Score (-0.5).
- *Hasil:* Di tabel `non_outliers_df`, sekarang ada kolom baru bernama `Cluster`.

Langkah 4: Persiapan Warna-Warni

```
cluster_colors = {0: '#1f77b4', 1: '#ff7f0e', 2: '#2ca02c', 3: '#d62728'}
colors = non_outliers_df['Cluster'].map(cluster_colors)
```

- Kita bikin kamus warna manual biar konsisten.
 - Kelompok 0 = Biru
 - Kelompok 1 = Oranye
 - Kelompok 2 = Hijau
 - Kelompok 3 = Merah
- `.map(...)` : Menerjemahkan kolom `Cluster` (angka 0,1,2,3) menjadi kode warna Hex.

Langkah 5: Menggambar Plot 3D

```
fig = plt.figure(figsize=(10, 10))
ax = fig.add_subplot(projection='3d')

scatter = ax.scatter(non_outliers_df['MonetaryValue'], ..., c=colors,
marker='o')
```

- Ini sama persis kayak kode plot 3D sebelumnya.
- Bedanya cuma satu: parameter `c=colors`.
- Artinya: "Warnai setiap titik sesuai dengan warna kelompoknya masing-masing."

Hasilnya: Kamu mendapatkan visualisasi di mana customer yang "mirip" (satu kelompok) warnanya sama dan posisinya bergerombol.

Jelas alurnya? Dari data mentah → Scaling → K-Means Prediksi → Tempel Label → Gambar.

ini "Contekan Singkat" (Cheat Sheet) logika RFM supaya kamu bisa menginterpretasikan angkanya dengan tajam:

Kunci Jawaban Intepretasi RFM

1. Recency (R) - Kebaruan

- **Definisi:** Jarak hari sejak belanja terakhir.
- **Angka KECIL (misal: 10, 20): BAGUS.** Artinya baru aja belanja. Customer aktif/hangat.
- **Angka BESAR (misal: 200, 300): BAHAYA.** Artinya udah lama banget ngilang. Customer dingin/hampir kabur.

2. Frequency (F) - Keseringan

- **Definisi:** Berapa kali datang belanja.
- **Angka BESAR: BAGUS.** Loyal, sering mampir.
- **Angka KECIL (1 atau 2): BIASA.** Cuma iseng atau customer baru.

3. Monetary (M) - Uang

- **Definisi:** Total duit yang dikasih ke kita.
- **Angka BESAR: BAGUS.** Sultan, penyumbang omzet.
- **Angka KECIL: BIASA.** Belanja dikit-dikit.

In [109...

```
# Hitung rata-rata tiap metrik RFM berdasarkan Cluster
cluster_summary = non_outliers_df.groupby('Cluster')[['MonetaryValue', 'Frequency',

# Hitung juga jumlah orang di setiap cluster (biar tau mana yang mayoritas)
cluster_counts = non_outliers_df['Cluster'].value_counts().reset_index()
cluster_counts.columns = ['Cluster', 'Count']

# Gabungkan keduanya
final_summary = cluster_summary.merge(cluster_counts, on='Cluster')

# Tampilkan dengan format angka 2 desimal biar rapi
print("=== RATA-RATA PROFIL PER KLASSTER ===")
display(final_summary.round(2))
```

```
=== RATA-RATA PROFIL PER KLASSTER ===
```

	Cluster	MonetaryValue	Frequency	Recency	Count
0	0	1308.62	3.91	49.73	914
1	1	384.54	1.43	251.17	902
2	2	417.95	1.64	54.07	1499
3	3	2436.09	7.24	33.89	494

In [113...

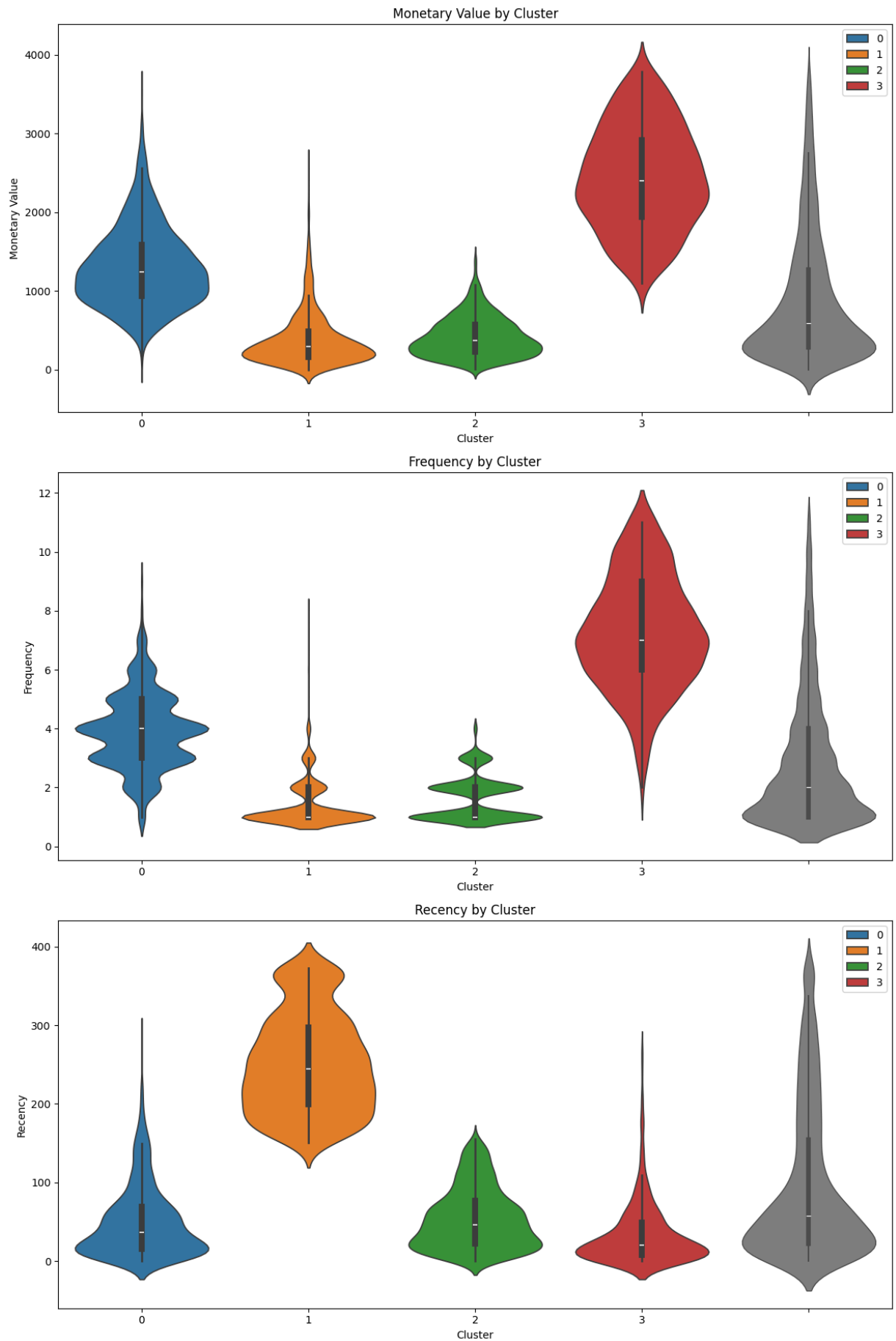
```
plt.figure(figsize=(12, 18))

plt.subplot(3, 1, 1)
sns.violinplot(x=non_outliers_df['Cluster'], y=non_outliers_df['MonetaryValue'], palette='magma',
               y=non_outliers_df['MonetaryValue'], color='gray', linewidth=1.0)
plt.title('Monetary Value by Cluster')
plt.ylabel('Monetary Value')

plt.subplot(3, 1, 2)
sns.violinplot(x=non_outliers_df['Cluster'], y=non_outliers_df['Frequency'], palette='magma',
               y=non_outliers_df['Frequency'], color='gray', linewidth=1.0)
plt.title('Frequency by Cluster')
plt.ylabel('Frequency')

plt.subplot(3, 1, 3)
sns.violinplot(x=non_outliers_df['Cluster'], y=non_outliers_df['Recency'], palette='magma',
               y=non_outliers_df['Recency'], color='gray', linewidth=1.0)
plt.title('Recency by Cluster')
plt.ylabel('Recency')

plt.tight_layout()
plt.show()
```

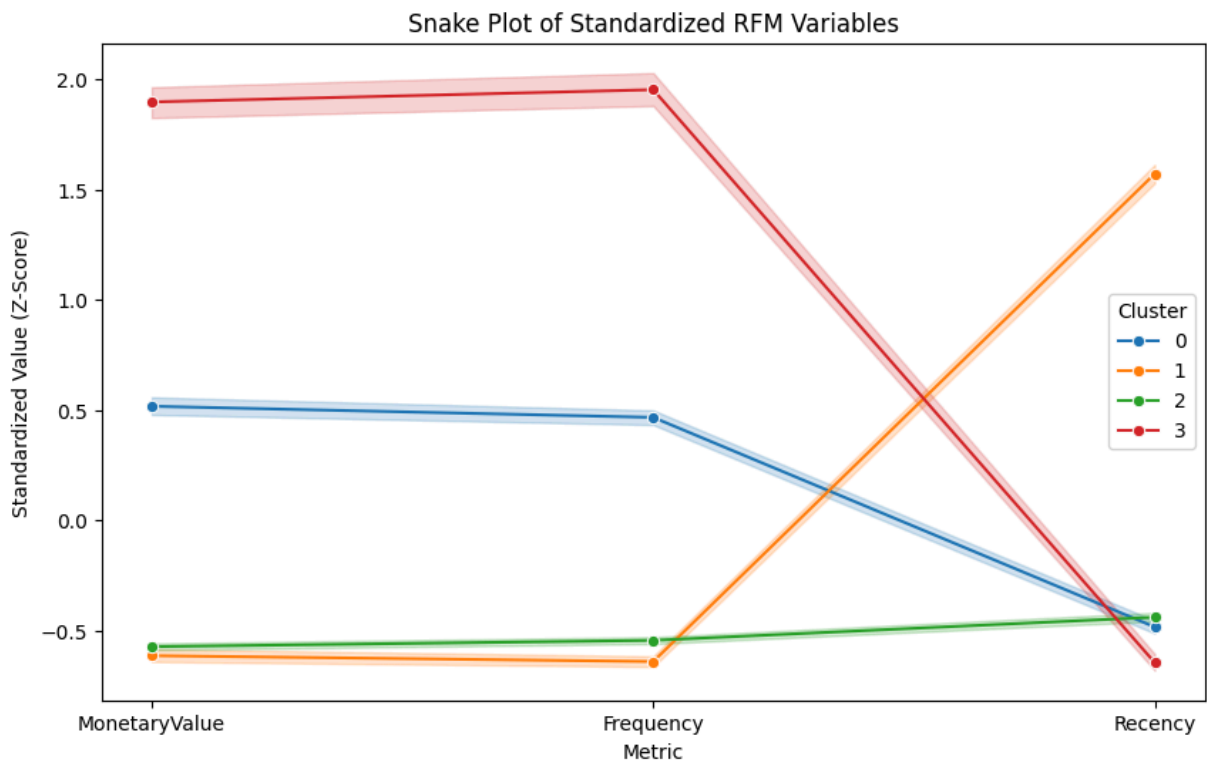


In [112...

```
# --- Kodingan Visualisasi Snake Plot ---
snake_data = scaled_data_df.copy()
snake_data['Cluster'] = cluster_labels

# Reset index biar indexnya jadi kolom biasa, TAPI kita gak usah panggil namanya sp
snake_melt = pd.melt(snake_data.reset_index(),
                     id_vars=['Cluster'], # Cukup Cluster aja sebagai identitas
                     value_vars=['MonetaryValue', 'Frequency', 'Recency'],
                     var_name='Metric', value_name='Value')

plt.figure(figsize=(10, 6))
sns.lineplot(data=snake_melt, x='Metric', y='Value', hue='Cluster', palette=cluster
plt.title('Snake Plot of Standardized RFM Variables')
plt.xlabel('Metric')
plt.ylabel('Standardized Value (Z-Score)')
plt.legend(title='Cluster')
plt.show()
```



5. Cluster Profiling & Strategic Insights

Pada tahap akhir ini, kita menerjemahkan angka statistik menjadi profil pelanggan yang nyata. Analisis ini didasarkan pada visualisasi **Snake Plot** dan **Summary Table**.



Catatan Kolaborasi: Interpretasi di bawah ini merupakan hasil sintesis dari analisis mandiri saya yang kemudian divalidasi dan dipertajam melalui diskusi dengan AI Assistant untuk mendapatkan terminologi bisnis yang tepat.

A. Analisis Awal (My Initial Hypothesis)

Sebelum melihat istilah teknis marketing, berikut adalah interpretasi saya berdasarkan data visual:

1. **Cluster 0 (Biru):** Performanya "Lumayan". Uang dan frekuensi oke, baru belanja juga. Punya potensi jadi seperti Cluster 3.
2. **Cluster 1 (Oranye):** Kelompok paling "Buruk". Jarang belanja, uang dikit, dan sudah lama sekali tidak muncul (Recency tinggi).
3. **Cluster 2 (Hijau):** Mirip Cluster 1 (belanja dikit), TAPI bedanya mereka baru saja belanja (Recency rendah). Kemungkinan pelanggan baru.
4. **Cluster 3 (Merah):** Kelompok "VIP". Paling sering belanja, uang paling banyak, dan aktif. Harus dijaga.

-> Versi Originalnya :

Oke kalau yang saya lihat disini, ada 4 kluster sesuai dengan analisis Kmeans yang sudah kita lakukan, berikut analisis atau interpretasi dari gambar tersebut

1. Kluster 0 : **MonetaryValue** lumayan, **Frequency** juga lumayan, serta **Recency** yang rendah
2. Kluster 1 : **MonetaryValue** rendah sekali dibandingkan dengan cluster lainnya, **Frequency** juga paling rendah, lalu **Recency** nya nilainya sangat tinggi. Kelompok ini bisa langsung diambil kesimpulan bahwa kelompok ini memiliki kriteria yang paling buruk.
3. Kluster 2 : **MonetaryValue** dan **Frequency** memiliki karakteristik yang hampir sama dengan Kluster 1, hanya tinggi sedikit saja. Namun **Recency** jauh lebih rendah dibandingkan Kluster 1 dan hampir sama dengan Kluster 0.
4. Kluster 3 : **MonetaryValue** dan **Frequency** memiliki nilai yang paling tinggi dan selisih jauh dibandingkan kelompok lainnya. Lalu memiliki nilai **Recency** yang sangat rendah.

Menurut saya, strategi yang bisa kita lakukan adalah (Action) :

- Kluster 0: Menurut saya ini memiliki potensial untuk bisa menjadi seperti karakteristik yang dimiliki oleh kluster 3. Jujur saya agak bingung strategi marketnya gimana, bisa diimpelmentasikan hal yang sama seperti kluster 3, namun lebih di personalized lagi, agar benar benar sering dan mengeluarkan banyak uang untuk toko kita. Kluster 0 diberikan perhatian lebih dibandingkan 3, karena kluster 3 menurut saya tanpa perlu benar benar se effort cluster 0, mereka akan tetap sering membeli di kita.
- Kluster 1: Re engage sih, kita coba deketin mereka lagi, memang ini adalah kelompok yang paling buruk secara karakteristik, namun mereka **Count** nya cukup banyak sehingga kita tidak bisa mengambil kesimpulan untuk membiarkan mereka. Mungkin

bisa kita lakukan dengan pemberian diskon, atau bilang kayak "sudah lama tidak belanja di kami, ini loh ada program A B C atau Diskon A B C" seperti itu

- Kluster 2: Seperti nya ini adalah pelanggan yang masih baru pada toko kita. Sehingga mereka masih belum tau nih atau mengenal mengenai toko kita. Kita coba untuk fokus building relationship dengan mereka, memberikan layanan customer yang baik. Memberikan juga insentif agar mendorong mereka untuk sering membeli. Kita bikin mereka bener bener ketergantungan dulu.
- Kluster 3 : Dia adalah kelompok VIP kita. Most loyal bisa dikatakan seperti itu, Dia sangat jauh lebih sering beli di toko kita dan paling banyak mengeluarkan uang untuk kita. Nah berarti kita setidaknya harus melakukan sesuatu yang bisa membuat mereka terasa di rekognisi oleh kita. Give reward, atau loyalty program seperti kartu member mungkin atau apa. Bisa juga kalau ada keluhan, mereka paling diprioritaskan dan semacamnya. Intinya recognize their loyalty to keep them engaged and satisfied.

B. Strategi Bisnis Terpadu (Refined Strategy)

Berdasarkan hipotesis di atas, berikut adalah pemetaan profil pelanggan (Persona) dan rekomendasi aksi strategis (*Actionable Insights*):

● Cluster 3: "Champions" (Para Juara)

- **Karakteristik:** *High Monetary, High Frequency, Low Recency.*
- **Profil:** Ini adalah pelanggan terbaik (Top Tier). Mereka berkontribusi paling besar terhadap omzet dan sangat loyal.
- **Strategi (Retention):**
 - Berikan layanan prioritas/VIP.
 - Akses awal (*Early Access*) untuk produk baru.
 - Jangan berikan diskon harga (karena mereka tidak sensitif harga), tapi berikan *Reward Points* atau hadiah eksklusif untuk menjaga ikatan emosional.

● Cluster 0: "Potential Loyalists" (Calon Pelanggan Setia)

- **Karakteristik:** *Average Monetary, Average Frequency, Low Recency.*
- **Profil:** Pelanggan aktif dengan potensi belanja yang belum maksimal. Mereka rutin berbelanja tetapi belum mencapai level "Sultan".
- **Strategi (Up-Selling & Cross-Selling):**
 - Tawarkan produk pelengkap (*Cross-sell*) untuk meningkatkan nilai keranjang belanja.
 - Berikan insentif ambang batas (misal: "Belanja sedikit lagi untuk dapat Gratis Ongkir/Bonus") untuk menaikkan Monetary mereka mendekati Cluster 3.

● Cluster 2: "New / Promising" (Pendatang Baru)

- **Karakteristik:** *Low Monetary, Low Frequency, Low Recency.*

- **Profil:** Pelanggan yang baru saja bergabung atau baru mencoba bertransaksi sedikit. Nilai transaksinya masih kecil, namun momentumnya bagus (baru aktif).
- **Strategi (Onboarding & Relationship Building):**
 - Fokus pada kepuasan pengalaman belanja pertama.
 - Edukasi mengenai *brand* dan katalog produk.
 - Berikan insentif kunjungan berikutnya (*Next Purchase Coupon*) untuk membangun kebiasaan (*habit*).

● Cluster 1: "Hibernating / Lost" (Tidur Panjang)

- **Karakteristik:** *Low Monetary, Low Frequency, High Recency*.
- **Profil:** Pelanggan yang sudah sangat lama tidak kembali (>250 hari). Kontribusi historis mereka juga rendah.
- **Strategi (Reactivation):**
 - Kirimkan kampanye *Win-Back* yang agresif (Diskon besar "Kami Rindu Anda").
 - **Catatan:** Jika kampanye re-aktivasi gagal, pertimbangkan untuk menghentikan biaya marketing ke segmen ini (*Churn*) agar efisiensi budget terjaga.



Visualisasi Pendukung: Snake Plot

Grafik *Snake Plot* di atas mempertegas perbedaan karakteristik ini melalui *Z-Score*:

- Garis **Merah (Cluster 3)** berada di puncak untuk Monetary & Frequency, namun di dasar untuk Recency (Sangat ideal).
- Garis **Oranye (Cluster 1)** melambung tinggi hanya di Recency, menandakan ketidakaktifan yang ekstrem.

UTANG DATA OUTLIER KTIA

In [115...

```
overlap_indices = monetary_outliers_df.index.intersection(frequency_outliers_df.index)

monetary_only_outliers = monetary_outliers_df.drop(overlap_indices)
frequency_only_outliers = frequency_outliers_df.drop(overlap_indices)
monetary_and_frequency_outliers = monetary_outliers_df.loc[overlap_indices]

monetary_only_outliers["Cluster"] = -1
frequency_only_outliers["Cluster"] = -2
monetary_and_frequency_outliers["Cluster"] = -3

outlier_clusters_df = pd.concat([monetary_only_outliers, frequency_only_outliers, monetary_and_frequency_outliers])
outlier_clusters_df
```

Out[115...

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	Recency	Cluster
9	12357.00	11229.99	1	2010-11-16 10:05:00	23	-1
25	12380.00	4782.84	4	2010-08-31 14:54:00	100	-1
42	12409.00	12346.62	4	2010-10-15 10:24:00	55	-1
48	12415.00	19468.84	4	2010-11-29 15:07:00	10	-1
61	12431.00	4145.52	11	2010-12-01 10:03:00	8	-1
...	...	...	...	...	...	...
4235	18223.00	7516.31	12	2010-11-17 12:20:00	22	-3
4236	18225.00	7545.14	15	2010-12-09 15:46:00	0	-3
4237	18226.00	6650.83	15	2010-11-26 15:51:00	13	-3
4241	18231.00	4791.80	23	2010-10-29 14:17:00	41	-3
4262	18260.00	7318.91	17	2010-11-30 12:25:00	9	-3

476 rows × 6 columns

In [118...

```
# Hitung rata-rata tiap metrik RFM berdasarkan Cluster
cluster_summary = outlier_clusters_df.groupby('Cluster')[['MonetaryValue', 'Frequency', 'Recency']].mean()

# Hitung juga jumlah orang di setiap cluster (biar tau mana yang mayoritas)
cluster_counts = outlier_clusters_df['Cluster'].value_counts().reset_index()
cluster_counts.columns = ['Cluster', 'Count']

# Gabungkan keduanya
final_summary = cluster_summary.merge(cluster_counts, on='Cluster')

# Tampilkan dengan format angka 2 desimal biar rapi
print("=== RATA-RATA PROFIL PER KLASER ===")
display(final_summary.round(2))
```

=== RATA-RATA PROFIL PER KLASER ===

	Cluster	MonetaryValue	Frequency	Recency	Count
0	-3	17147.66	25.87	14.45	226
1	-2	2734.69	15.04	23.08	53
2	-1	6498.45	7.19	47.91	197

In [117...

```
cluster_colors = {-1: '#9467bd',
                  -2: '#8c564b',
                  -3: '#e377c2'}

plt.figure(figsize=(12, 18))

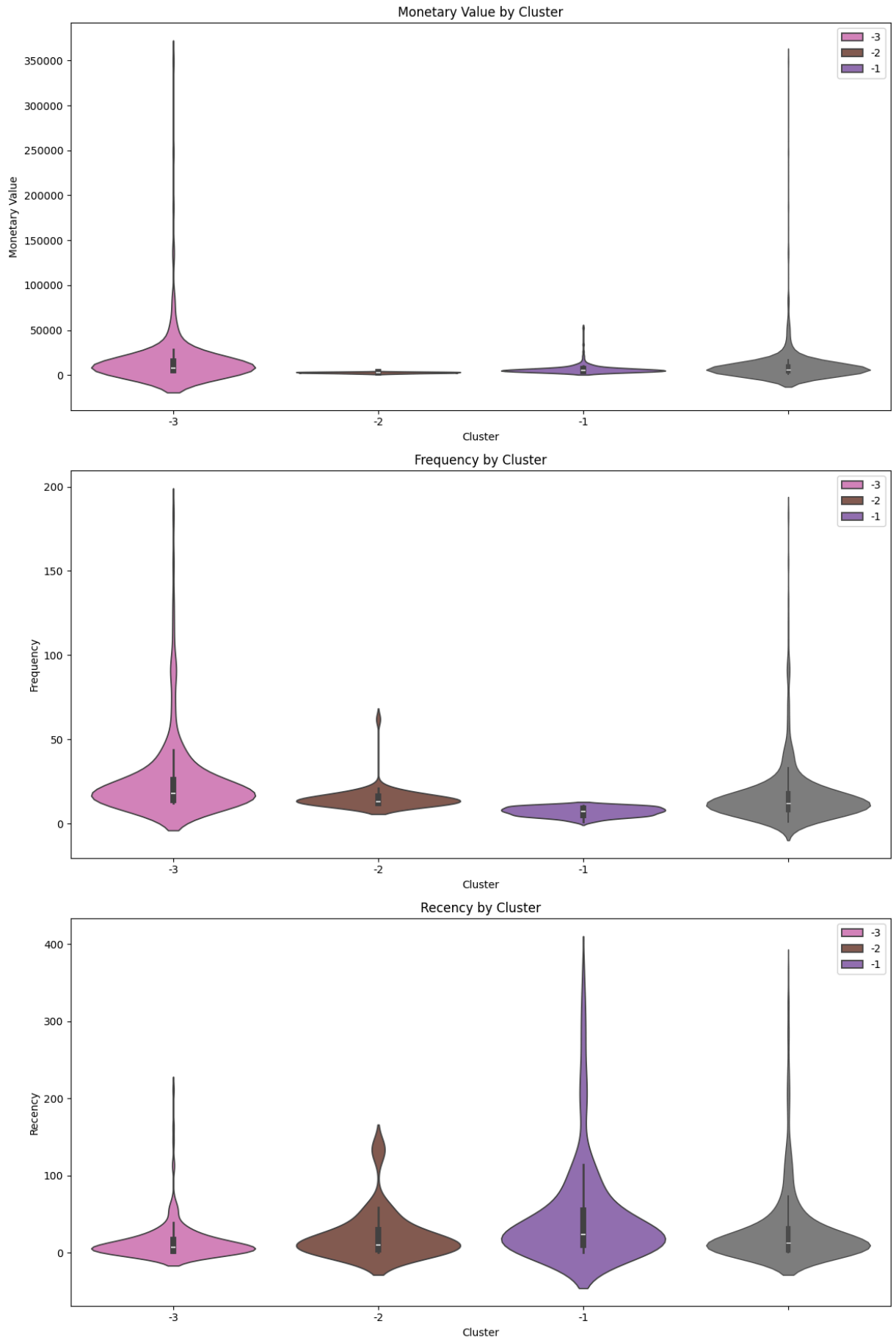
plt.subplot(3, 1, 1)
```

```
sns.violinplot(x=outlier_clusters_df['Cluster'], y=outlier_clusters_df['MonetaryVal
sns.violinplot(y=outlier_clusters_df['MonetaryValue'], color='gray', linewidth=1.0)
plt.title('Monetary Value by Cluster')
plt.ylabel('Monetary Value')

plt.subplot(3, 1, 2)
sns.violinplot(x=outlier_clusters_df['Cluster'], y=outlier_clusters_df['Frequency']
sns.violinplot(y=outlier_clusters_df['Frequency'], color='gray', linewidth=1.0)
plt.title('Frequency by Cluster')
plt.ylabel('Frequency')

plt.subplot(3, 1, 3)
sns.violinplot(x=outlier_clusters_df['Cluster'], y=outlier_clusters_df['Recency'],
sns.violinplot(y=outlier_clusters_df['Recency'], color='gray', linewidth=1.0)
plt.title('Recency by Cluster')
plt.ylabel('Recency')

plt.tight_layout()
plt.show()
```



In [121...

```

# 1. Scaling Data Outlier
scaler_outlier = StandardScaler()
scaled_outliers = scaler_outlier.fit_transform(outlier_clusters_df[["MonetaryValue", "Frequency", "Recency"])

# 2. Bikin DataFrame hasil scaling
scaled_outliers_df = pd.DataFrame(scaled_outliers,
                                   index=outlier_clusters_df.index,
                                   columns=["MonetaryValue", "Frequency", "Recency"])

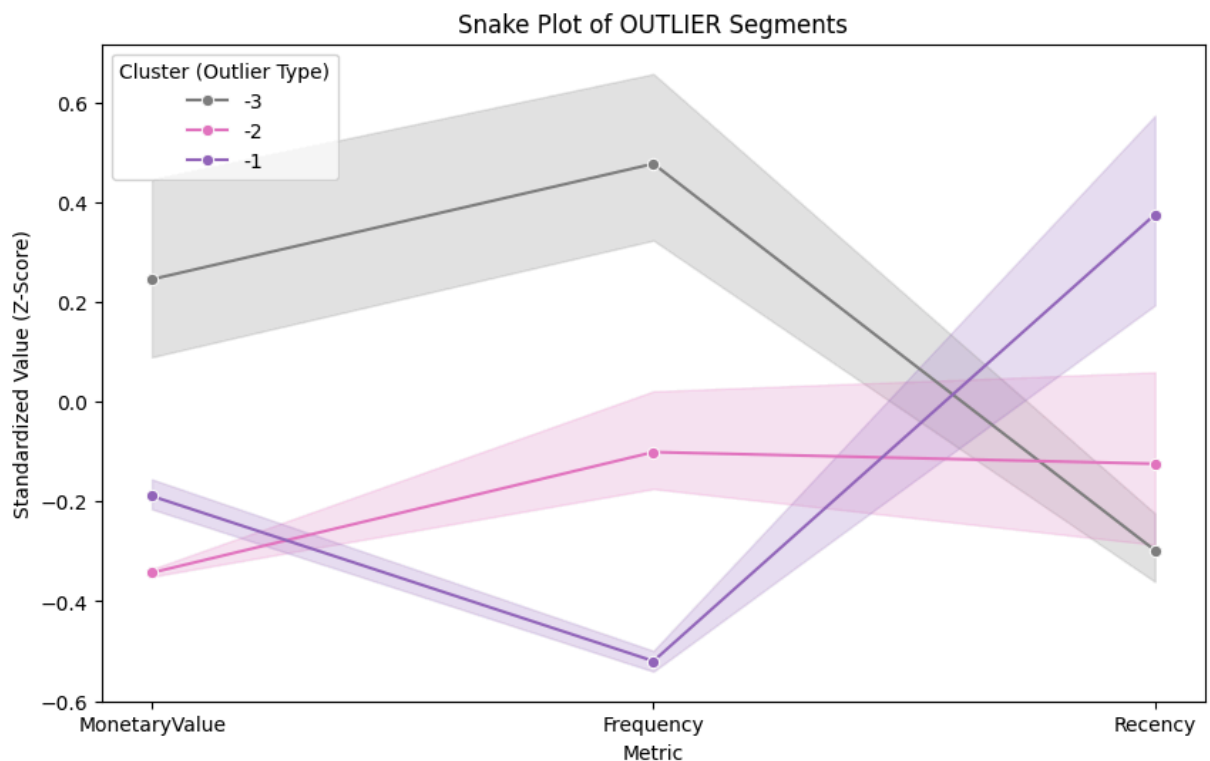
# 3. Tempel Label Clusternya (-1, -2, -3)
scaled_outliers_df['Cluster'] = outlier_clusters_df['Cluster']

# 4. Melt Data (Bentuk Ular)
snake_melt_outlier = pd.melt(scaled_outliers_df.reset_index(),
                              id_vars=['Cluster'],
                              value_vars=['MonetaryValue', 'Frequency', 'Recency'],
                              var_name='Metric', value_name='Value')

# 5. Gambar!
# Kita pakai warna khusus biar beda (Ungu, Pink, Abu)
outlier_palette = {-1: '#9467bd', -2: '#e377c2', -3: '#7f7f7f'}

plt.figure(figsize=(10, 6))
sns.lineplot(data=snake_melt_outlier, x='Metric', y='Value', hue='Cluster', palette=outlier_palette)
plt.title('Snake Plot of OUTLIER Segments')
plt.xlabel('Metric')
plt.ylabel('Standardized Value (Z-Score)')
plt.legend(title='Cluster (Outlier Type)')
plt.show()

```



Analisis Segmen Outlier (High-Value Customers)

Berdasarkan *Summary Table* dan *Snake Plot* khusus outlier, kita dapat membedah karakteristik 3 kelompok pelanggan ekstrem ini:

1. Cluster -3: "The Absolute VVIPs" (Overlap Group)

Ini adalah irisan dari *High Monetary* DAN *High Frequency*.

- **Data:**
 - **Monetary:** Rata-rata **17.147** (Tertinggi mutlak).
 - **Frequency:** Rata-rata **25 kali** (Sangat sering).
 - **Recency:** **14 hari** (Sangat aktif).
- **Analisis:** Ini adalah **aset terbesar perusahaan**. Jumlahnya ada 226 orang. Mereka belanja sangat sering dengan nilai transaksi yang sangat besar. Hampir bisa dipastikan ini adalah **Mitra Bisnis Besar** atau pelanggan korporat yang sangat loyal.
- **Rekomendasi:** *Personalized Service*. Jangan perlakukan mereka dengan bot. Berikan *Key Account Manager* khusus untuk menangani pesanan mereka.

2. Cluster -1: "Big Spenders" (Monetary Only)

- **Data:**
 - **Monetary:** Rata-rata **6.498** (Tinggi).
 - **Frequency:** Rata-rata **7 kali** (Paling rendah di antara para outlier).
 - **Recency:** **48 hari** (Agak lama dibanding outlier lain).
- **Analisis:** Mereka mengeluarkan banyak uang, tapi tidak terlalu sering datang. Pola ini mengindikasikan pembelian **barang mahal dalam sekali transaksi** atau pembelian stok besar dalam interval waktu yang jarang.
- **Rekomendasi:** Fokus pada *Replenishment Reminder* (Pengingat stok ulang). Tawarkan produk premium atau *bulk-buy* dengan diskon volume.

3. Cluster -2: "High Frequency Traders" (Frequency Only)

- **Data:**
 - **Monetary:** Rata-rata **2.734** (Paling "kecil" di antara para outlier, tapi masih jauh di atas rata-rata populasi umum).
 - **Frequency:** Rata-rata **15 kali** (Sangat sering).
 - **Recency:** **23 hari** (Aktif).
- **Analisis:** Mereka sangat rajin datang, tapi nilai per transaksinya tidak sebesar dua kelompok lainnya. Kemungkinan besar ini adalah **Reseller kecil** atau pengecer yang rutin mengisi stok sedikit demi sedikit.

- **Rekomendasi:** Jaga ketersediaan stok (*Inventory Availability*). Pastikan barang yang sering mereka cari selalu *ready* agar mereka tidak pindah ke *supplier* lain.

Visualisasi Snake Plot Outlier

Grafik garis (Snake Plot) mengonfirmasi hierarki ini:

- **Garis Abu-abu (Cluster -3):** Mendominasi di posisi paling atas pada semua metrik (Monetary & Frequency tinggi, Recency rendah/bagus). Ini adalah profil pelanggan yang sempurna.
- **Garis Ungu (Cluster -1):** Menonjol di Monetary, tapi anjlok di Frequency.
- **Garis Pink (Cluster -2):** Menonjol di Frequency, tapi paling bawah di Monetary (untuk ukuran outlier).

Technical Deep Dive: Memahami Visualisasi

Selain melihat tren naik-turun, kita juga perlu memahami makna **dimensi bentuk** pada grafik di atas untuk menilai konsistensi data dalam setiap segmen.

1. Snake Plot: Area Bayangan (*Confidence Interval*)

Pada grafik garis (Snake Plot), area transparan (bayangan) di sekitar garis utama merepresentasikan **Confidence Interval 95%**.

- **Interpretasi Bisnis:** Area ini menunjukkan seberapa "kompak" perilaku pelanggan dalam satu grup.
- **Pita Tipis (Sempit):** Menandakan **Konsistensi Tinggi**. Mayoritas pelanggan di klaster tersebut memiliki perilaku yang sangat mirip. Prediksi kita terhadap perilaku mereka sangat akurat.
 - *Contoh:* Cluster -3 pada metrik Frequency memiliki pita yang relatif tipis, artinya para VVIP ini konsisten sering berbelanja.
- **Pita Lebar:** Menandakan **Variasi Tinggi**. Perilaku pelanggan di klaster ini lebih beragam (ada yang sangat tinggi, ada yang rendah). Strategi marketing untuk grup ini mungkin perlu lebih fleksibel.
 - *Contoh:* Cluster -1 pada metrik Recency memiliki pita yang sangat lebar. Artinya, meskipun mereka rata-rata belanja 48 hari lalu, ada yang baru belanja kemarin dan ada yang sudah lama hilang.

2. Violin Plot: Bentuk Gendut vs Kurus (*Kernel Density*)

Violin plot menggambarkan **distribusi kepadatan probabilitas** data.

- **Bagian Gendut (Lebar):** Menunjukkan di mana **mayoritas populasi** berkumpul (*High Density*).

- *Contoh:* Jika bagian bawah violin gendut, berarti kebanyakan pelanggan memiliki nilai rendah.
- **Bagian Kurus (Panjang):** Menunjukkan adanya data yang menyebar jauh (*Long Tail*).
- **Analisis Cluster -3 (VVIP):**
 - Bentuk violin yang memanjang pada *Monetary* menunjukkan bahwa meskipun semua anggota klaster ini adalah "Sultan", masih ada hierarki kekayaan yang luas di antara mereka (dari yang kaya hingga yang *ultra-kaya*).

Insight Teknis: Memahami variasi ini penting agar kita tidak terjebak pada angka "Rata-rata" semata. Rata-rata bisa menipu jika variasi datanya (lebar pitanya) terlalu besar.

```
In [ ]: cluster_labels = {
    0: "RETAIN",
    1: "RE-ENGAGE",
    2: "NURTURE",
    3: "REWARD",
    -1: "PAMPER", # (Manjakan, karena belanja mahal tapi jarang).
    -2: "UPSELL", # (Tawarkan barang lebih mahal, karena sering beli tapi murah).
    -3: "DELIGHT" # (Sangat Bahagiakan, karena ini VVIP Absolute).
}
```

```
In [ ]: full_clustering_df = pd.concat([non_outliers_df, outlier_clusters_df])
# pd.concat: Menggabungkan tabel non_outliers (mayoritas) dengan tabel outliers (el
full_clustering_df
```

```
Out[ ]:
```

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	Recency	Cluster
0	12346.00	169.36	2	2010-06-28 13:53:00	164	1
1	12347.00	1323.32	2	2010-12-07 14:57:00	2	0
2	12348.00	221.16	1	2010-09-27 14:59:00	73	2
3	12349.00	2221.14	2	2010-10-28 08:23:00	42	0
4	12351.00	300.93	1	2010-11-29 15:23:00	10	2
...	...	...	...	...	...	...
4235	18223.00	7516.31	12	2010-11-17 12:20:00	22	-3
4236	18225.00	7545.14	15	2010-12-09 15:46:00	0	-3
4237	18226.00	6650.83	15	2010-11-26 15:51:00	13	-3
4241	18231.00	4791.80	23	2010-10-29 14:17:00	41	-3
4262	18260.00	7318.91	17	2010-11-30 12:25:00	9	-3

4285 rows × 6 columns

```
In [126... full_clustering_df["ClusterLabel"] = full_clustering_df["Cluster"].map(cluster_label_map)
full_clustering_df
```

```
Out[126... 
```

	Customer ID	MonetaryValue	Frequency	LastInvoiceDate	Recency	Cluster	ClusterLabel
0	12346.00	169.36	2	2010-06-28 13:53:00	164	1	RE-ENGAGE
1	12347.00	1323.32	2	2010-12-07 14:57:00	2	0	RETAIL
2	12348.00	221.16	1	2010-09-27 14:59:00	73	2	NURTURE
3	12349.00	2221.14	2	2010-10-28 08:23:00	42	0	RETAIL
4	12351.00	300.93	1	2010-11-29 15:23:00	10	2	NURTURE
...	...	...	...	...	...	...	...
4235	18223.00	7516.31	12	2010-11-17 12:20:00	22	-3	DELIGHT
4236	18225.00	7545.14	15	2010-12-09 15:46:00	0	-3	DELIGHT
4237	18226.00	6650.83	15	2010-11-26 15:51:00	13	-3	DELIGHT
4241	18231.00	4791.80	23	2010-10-29 14:17:00	41	-3	DELIGHT
4262	18260.00	7318.91	17	2010-11-30 12:25:00	9	-3	DELIGHT

4285 rows × 7 columns



Visualisation

```
In [127... cluster_counts = full_clustering_df['ClusterLabel'].value_counts()
full_clustering_df["MonetaryValue per 100 pounds"] = full_clustering_df["MonetaryValue"] / 100
feature_means = full_clustering_df.groupby('ClusterLabel')[['Recency', 'Frequency', 'MonetaryValue per 100 pounds']].mean()

fig, ax1 = plt.subplots(figsize=(12, 8))

sns.barplot(x=cluster_counts.index, y=cluster_counts.values, ax=ax1, palette='viridis')
ax1.set_ylabel('Number of Customers', color='b')
ax1.set_title('Cluster Distribution with Average Feature Values')

ax2 = ax1.twinx()
```

```
sns.lineplot(data=feature_means, ax=ax2, palette='Set2', marker='o')  
ax2.set_ylabel('Average Value', color='g')  
  
plt.show()
```

